BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC ĐIỆN LỰC
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CHUYÊN ĐỀ HỌC PHẦN

HỆ THỐNG IoT VÀ ỨNG DỤNG

THIẾT KẾ HỆ THỐNG GIÁM SÁT VÀ ĐIỀU KHIỂN NHÀ THÔNG MINH TÍCH HỢP TRỢ LÝ AI

Giảng viên hướng dẫn ThS. Phạm Văn Hiệp

Sinh viên thực hiện

Họ và tên	Mã sinh viên	Lớp
Nguyễn Thùy Linh	23810310378	D18CNPM5
Lê Hải Nam	23810310377	D18CNPM5
Nguyễn Xuân Đạt	23810310361	D18CNPM5
Vũ Tiến Mạnh	23810310375	D18CNPM5
Đinh Việt Dũng	23810310370	D18CNPM5
Nguyễn Đắc Tuấn Nghĩa	23810310343	D18CNPM5

Hà Nội, tháng 10 năm 2026

LỜI CẢM ƠN

Chúng em xin trân trọng cảm ơn ThS. Phạm Văn Hiệp đã giảng dạy và hướng dẫn chúng em trong học phần Hệ thống IoT và ứng dụng. Những kiến thức về cảm biến, kết nối thiết bị, giao thức truyền thông và xây dựng ứng dụng đã giúp nhóm hình thành và triển khai hệ thống nhà thông minh trong báo cáo này.

Chúng em cảm ơn các thầy cô Khoa Công nghệ Thông tin, Trường Đại học Điện lực đã trang bị nền tảng kiến thức lập trình, cơ sở dữ liệu và mạng máy tính. Trong quá trình thực hiện, các thành viên trong nhóm cùng trao đổi, lắp ráp mô hình, kiểm tra chức năng và hoàn thiện phần mềm để kết nối các nội dung đã học với một bài toán cụ thể.

Báo cáo trình bày quá trình thiết kế, triển khai và đánh giá hệ thống giám sát phòng khách sử dụng ESP32, MQTT, Node.js, MySQL và giao diện web, đồng thời tích hợp trợ lý AI chạy cục bộ. Nhóm mong nhận được ý kiến góp ý của giảng viên để cải thiện độ ổn định, tính dễ sử dụng và khả năng mở rộng của hệ thống.

Nhóm sinh viên lớp D18CNPM5

TÓM TẮT ĐỀ TÀI

Đề tài xây dựng một mô hình nhà thông minh ở phạm vi phòng khách, trong đó ESP32 thu nhận nhiệt độ, độ ẩm, chuyển động và trạng thái ánh sáng; điều khiển relay, đèn LED và còi; đồng thời trao đổi dữ liệu với máy chủ qua WiFi và MQTT. Hệ thống kết hợp tự động hóa tại thiết bị với giám sát, lưu trữ và tương tác trên trình duyệt.

Phần máy chủ sử dụng Node.js và Express để cung cấp REST API, MySQL để lưu lịch sử và thói quen người dùng, WebSocket để cập nhật dữ liệu cho dashboard React. Trợ lý AI có hai nhánh xử lý: nhận diện lệnh đèn và còi bằng quy tắc; trả lời câu hỏi về ngôi nhà bằng mô hình llama3.1 chạy trên Ollama với ngữ cảnh lấy từ cơ sở dữ liệu.

Dữ liệu lịch sử của hai phiên chạy ngày 01/09/2026 và 08/09/2026 gồm 2.077 bản ghi MQTT, trong đó có 1.950 bản ghi trạng thái và 127 bản ghi lệnh. Có 304 bản ghi nhiệt độ trong khoảng 29,4–35,0°C và 303 bản ghi độ ẩm trong khoảng 54–88%. Các con số này mô tả tập dữ liệu đã lưu, không thay thế phép hiệu chuẩn cảm biến hay phép đo tỷ lệ tiết kiệm điện.

Nhóm đã thử nghiệm thành công các chức năng giám sát cảm biến, điều khiển tự động và thủ công, lưu lịch sử, cập nhật giao diện và hỏi đáp AI. Các màn hình web, ảnh mô hình và dữ liệu MQTT minh họa cho từng kết quả triển khai. Lần kiểm tra ngày 15/10/2026 tiếp tục xác nhận khả năng kết nối, truy vấn và phản hồi của các dịch vụ phần mềm. Từ kết quả đạt được, báo cáo phân tích những giới hạn còn lại và đề xuất các bước phát triển tiếp theo.

Từ khóa: nhà thông minh, ESP32, ESPHome, MQTT, Node.js, React, MySQL, Ollama.

MỤC LỤC

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI 1
Lý do chọn đề tài 1
Mục tiêu của hệ thống 1
Đối tượng và phạm vi thực hiện 2
Phương pháp thực hiện 2
Cấu trúc báo cáo 2
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 3
Mô hình chức năng của hệ thống IoT 3
ESP32 và nền tảng ESPHome 3
Các cảm biến và cơ cấu chấp hành 4
WiFi và giao tiếp trong mạng nội bộ 5
Giao thức MQTT 6
REST API và WebSocket 6
Node.js và cơ sở dữ liệu MySQL 7
Giao diện React và trợ lý AI cục bộ 7
CHƯƠNG 3 PHÂN TÍCH THIẾT KẾ VÀ TRIỂN KHAI 8
Yêu cầu chức năng 8
Yêu cầu vận hành 8
Kiến trúc tổng thể 9
Thiết kế phần cứng và phân bố chân 10
Thiết kế firmware và chế độ hoạt động 12
Thiết kế cây topic và payload MQTT 14
Thiết kế cơ sở dữ liệu 14
Xử lý dữ liệu tại backend 16
Thiết kế các REST API 16
Gợi ý môi trường và phát hiện bất thường 17
Thiết kế trợ lý AI 18

Thiết kế mạng và phân bố các dịch vụ 19
Vận hành dịch vụ và kiểm tra khi kết nối gián đoạn 25
Giao diện web và tổ chức các chức năng 26
Công cụ cấu hình và kiểm chứng kết nối 33
CHƯƠNG 4 KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ 37
Môi trường và phạm vi đánh giá 37
Kết quả thống kê dữ liệu lịch sử 37
Đặc điểm số liệu nhiệt độ và độ ẩm 39
Minh họa luồng lệnh và phản hồi từ dữ liệu cũ 39
Kết quả thử nghiệm chức năng của mô hình 40
Kiểm tra lại phần mềm ngày 15 tháng 10 năm 2026 41
Kiểm tra các ngưỡng và nhận diện câu lệnh 42
Kết quả hỏi đáp AI 43
Ưu điểm của hệ thống 43
Giới hạn và hướng hoàn thiện 44
KẾT LUẬN 46
TÀI LIỆU THAM KHẢO 47
PHỤ LỤC A CẤU HÌNH ESPHOME 48
PHỤ LỤC B HƯỚNG DẪN CHẠY VÀ ĐỐI CHIẾU 50
Khởi động các dịch vụ 50
Địa chỉ dùng khi chạy trên máy chủ 50
Ví dụ nội dung yêu cầu và phản hồi 50
Truy vấn đối chiếu tập dữ liệu lịch sử 51
Các lưu ý khi mở rộng cấu hình 51

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên gọi	Ý nghĩa trong đề tài
IoT	Internet of Things	Kết nối thiết bị vật lý với ứng dụng
MCU	Microcontroller Unit	Vi điều khiển ESP32
GPIO	General Purpose Input Output	Chân tín hiệu vào và ra
PIR	Passive Infrared	Cảm biến phát hiện chuyển động
LDR	Light Dependent Resistor	Phần tử quang trở trong module ánh sáng
MQTT	MQTT	Giao thức truyền bản tin publish/subscribe
LAN	Local Area Network	Mạng nội bộ
TCP	Transmission Control Protocol	Giao thức vận chuyển
HTTP	Hypertext Transfer Protocol	Giao tiếp yêu cầu và phản hồi
REST	Representational State Transfer	Kiểu tổ chức API
API	Application Programming Interface	Giao diện lập trình ứng dụng
WS	WebSocket	Kết nối đẩy dữ liệu tới dashboard
JSON	JavaScript Object Notation	Dạng biểu diễn dữ liệu
OTA	Over The Air	Cập nhật firmware qua mạng
LLM	Large Language Model	Mô hình ngôn ngữ dùng cho hỏi đáp
AI	Artificial Intelligence	Trí tuệ nhân tạo

DANH MỤC HÌNH ẢNH

Hình 2.1 Các thành phần phần cứng sử dụng trong mô hình 5
Hình 3.1 Sơ đồ kiến trúc và giao tiếp giữa các khối 9
Hình 3.2 Mô hình lắp ráp phiên bản đầu 11
Hình 3.3 Mô hình bổ sung LED và còi ở phiên bản sau 11
Hình 3.4 Lưu đồ xử lý sự kiện PIR theo cấu hình hiện tại 13
Hình 3.5 Cấu trúc hai bảng dữ liệu chính 15
Hình 3.6 Luồng nhận diện lệnh và hỏi đáp của chatbot 19
Hình 3.7 Sơ đồ mạng LAN và vị trí các dịch vụ trên máy chủ 20
Hình 3.8 Hai luồng trao đổi trạng thái cảm biến và lệnh điều khiển 23
Hình 3.9 Màn hình Bảng điều khiển của website nhà thông minh 28
Hình 3.10 Biểu đồ nhiệt độ và độ ẩm với dữ liệu lịch sử 29
Hình 3.11 Khu vực điều khiển relay và chế độ vận hành 29
Hình 3.12 Trang Thiết bị sử dụng: tổng quan dự án và luồng hoạt động 30
Hình 3.13 Màn hình quản lý thói quen theo danh mục 31
Hình 3.14 Giao diện Trợ lý AI với ô nhập và các câu hỏi gợi ý 32
Hình 3.15 Cấu hình kết nối broker cục bộ trên MQTT Explorer 34
Hình 3.16 Nhánh topic phòng khách và dữ liệu ghi nhận trên MQTT Explorer 35
Hình 3.17 Giao diện ESPHome Device Builder dùng để cấu hình nút ESP32 36

DANH MỤC BẢNG BIỂU

Bảng 2.1 Các khối chức năng trong mô hình 3
Bảng 3.1 Các yêu cầu chức năng 8
Bảng 3.2 Phân bố chân theo cấu hình ESPHome 10
Bảng 3.3 Hành động tự động theo sự kiện 13
Bảng 3.4 Các topic trạng thái và lệnh 14
Bảng 3.5 Các trường trong bảng cam_bien_data 15
Bảng 3.6 Các trường trong bảng thoi_quen_nguoi_dung 15
Bảng 3.7 Các API của hệ thống 17
Bảng 3.8 Các ngưỡng gợi ý trong mã nguồn 18
Bảng 3.9 Phân bố dịch vụ, cổng và phía sử dụng 21
Bảng 3.10 Trình tự khởi động và dấu hiệu sẵn sàng 25
Bảng 3.11 Khoanh vùng lỗi mạng và dịch vụ 26
Bảng 4.1 Các nguồn dùng để đánh giá 37
Bảng 4.2 Số bản ghi theo topic trong hai phiên lịch sử 38
Bảng 4.3 Thống kê mô tả nhiệt độ và độ ẩm 39
Bảng 4.4 Một số sự kiện lệnh và trạng thái ngày 08 tháng 09 39
Bảng 4.5 Tổng hợp các kịch bản chức năng đã thử nghiệm 40
Bảng 4.6 Kết quả kiểm tra dịch vụ phần mềm 41
Bảng 4.7 Các trường hợp kiểm tra logic 42
Bảng 4.8 Các hướng phát triển ưu tiên 45

CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI

Lý do chọn đề tài

Một hệ thống nhà thông minh cần giúp người sử dụng trả lời hai câu hỏi thiết thực: không gian trong nhà đang ở trạng thái nào và có thể tác động đến thiết bị bằng cách nào. Với một phòng khách, nhu cầu đó được thể hiện qua việc theo dõi nhiệt độ, độ ẩm, sự xuất hiện của chuyển động, trạng thái ánh sáng và khả năng bật tắt các đầu ra. Đề tài được xây dựng từ những nhu cầu cụ thể này để kết nối kiến thức IoT với một sản phẩm có thể quan sát và thao tác trực tiếp.

Nhóm lựa chọn mô hình phòng khách vì phạm vi này đủ rõ để xây dựng và kiểm thử một chuỗi xử lý hoàn chỉnh. DHT11 cung cấp hai đại lượng môi trường; PIR tạo sự kiện chuyển động; module LDR cung cấp tín hiệu ánh sáng dạng nhị phân. Relay, LED và còi giúp thể hiện kết quả điều khiển trên mô hình thật, qua đó kiểm tra được quan hệ giữa cảm biến, chương trình nhúng và ứng dụng giám sát.

Bên cạnh các thao tác truyền thống, nhóm bổ sung trợ lý AI để người dùng hỏi về dữ liệu đang có và gửi một số lệnh ngắn. Phần mở rộng này tạo cơ hội nghiên cứu cách cung cấp ngữ cảnh thực tế cho mô hình ngôn ngữ, đồng thời giữ những lệnh điều khiển đơn giản trong nhánh xử lý xác định. Nhờ đó, đề tài bao quát được cả phần cứng, mạng, phần mềm và tương tác người dùng.

Mục tiêu của hệ thống

Mục tiêu tổng quát là xây dựng hệ thống giám sát và điều khiển nhà thông minh hoạt động trong mạng nội bộ, cho phép theo dõi trạng thái phòng khách qua trình duyệt và duy trì các phản ứng tự động tại ESP32. Máy chủ tập trung vào lưu trữ, cung cấp dữ liệu, tạo gợi ý và hỗ trợ hỏi đáp.

Về thu thập dữ liệu, hệ thống phải đọc được nhiệt độ, độ ẩm, chuyển động và ánh sáng, tổ chức bản tin theo từng topic MQTT và chuyển được dữ liệu về cơ sở dữ liệu. Về điều khiển, hệ thống phải thực hiện được logic tự động theo PIR và ánh sáng, tiếp nhận lệnh bật tắt relay, LED, còi và chuyển đổi chế độ thủ công.

Về ứng dụng, dashboard phải hiển thị các giá trị gần nhất, biểu đồ dữ liệu, trạng thái kết nối và công cụ điều khiển. Hệ thống cần quản lý các ghi chú thói quen để làm ngữ cảnh tham chiếu và cung cấp chatbot hỏi đáp dựa trên dữ liệu cảm biến, gợi ý, bất thường và thói quen đã lưu.

Đối tượng và phạm vi thực hiện

Đối tượng nghiên cứu là chuỗi kết nối giữa một nút ESP32, các cảm biến và cơ cấu chấp hành trong mô hình phòng khách, dịch vụ MQTT, máy chủ ứng dụng và trình duyệt. Mô hình có một nút điều khiển trung tâm ở phía thiết bị; các dịch vụ xử lý và lưu trữ được triển khai trên máy tính trong mạng LAN.

Phạm vi điều khiển trực tiếp gồm một kênh relay ở GPIO26, LED ở GPIO25 và còi ở GPIO33. Mặc dù phần cứng dùng module relay hai kênh, cấu hình hiện tại chỉ khai báo một kênh relay. Cảm biến ánh sáng được đọc qua ngõ ra số ở GPIO34; hệ thống chưa đo cường độ ánh sáng theo đơn vị lux.

Đề tài tập trung vào mô hình học tập và kiểm chứng các luồng chức năng. Việc mở rộng nhiều phòng, xây dựng tài khoản người dùng, triển khai truy cập từ Internet, hiệu chuẩn cảm biến với thiết bị chuẩn và đo công suất tiêu thụ là các hướng phát triển tiếp theo. Các thao tác ghi thói quen hiện tại chưa tạo ra lịch tự động thực thi.

Phương pháp thực hiện

Nhóm tiếp cận bài toán theo từng khối chức năng: lựa chọn cảm biến và chân kết nối; xây dựng cấu hình ESPHome; tổ chức truyền thông MQTT; triển khai máy chủ, cơ sở dữ liệu và dashboard; sau đó tích hợp chatbot. Cách làm này giúp kiểm tra riêng từng khối trước khi ghép thành hệ thống.

Việc đánh giá kết hợp ba nguồn: cấu hình và mã nguồn triển khai; bản ghi dữ liệu từ các phiên chạy đã có; thử nghiệm lại các dịch vụ phần mềm. Trạng thái phần cứng được đối chiếu với topic phản hồi, thay vì chỉ suy ra từ việc API đã nhận một yêu cầu điều khiển. Các phép kiểm tra logic dùng các giá trị ở hai phía của ngưỡng để tránh bỏ sót sai lệch tại biên.

Kết quả định lượng được thống kê từ dữ liệu MySQL. Khi phân tích, nhóm tách bản tin trạng thái khỏi bản tin lệnh và tách dữ liệu lịch sử khỏi dữ liệu retained nhận lại sau khi máy chủ kết nối. Cách phân loại này giúp diễn giải đúng số bản ghi và tránh coi các bản tin phát lại là phép đo mới.

Cấu trúc báo cáo

Báo cáo gồm bốn chương: tổng quan đề tài; cơ sở lý thuyết và công nghệ; phân tích thiết kế và triển khai; kết quả thử nghiệm và đánh giá. Phần cuối gồm kết luận, tài liệu tham khảo và phụ lục cấu hình, vận hành.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

Mô hình chức năng của hệ thống IoT

Trong đề tài, hệ thống được mô tả theo bốn nhóm chức năng: cảm nhận và chấp hành; kết nối mạng; xử lý và lưu trữ; ứng dụng. Đây là cách phân chia để giải thích vai trò của các khối trong mô hình, không phải phép đồng nhất với bảy lớp OSI. TCP được xem là giao thức vận chuyển dùng bên dưới các giao thức ứng dụng.

Ở phía thiết bị, ESP32 đọc tín hiệu vật lý và thực hiện logic điều khiển tại chỗ. Phía mạng dùng WiFi và mạng IP để đưa bản tin đến broker. Phía xử lý gồm broker, backend, MySQL và Ollama. Phía ứng dụng là dashboard trên trình duyệt. Một trạng thái cảm biến có thể đi xuyên qua cả bốn nhóm, trong khi thao tác tự động tại ESP32 có thể diễn ra mà không chờ xử lý từ máy chủ.

Bảng 2.1 Các khối chức năng trong mô hình

Khối	Thành phần	Vai trò
Cảm nhận và chấp hành	ESP32, DHT11, PIR, LDR, relay, LED, còi	Thu tín hiệu và tác động tại mô hình
Kết nối	WiFi, IP, TCP, MQTT	Truyền trạng thái và lệnh
Xử lý và lưu trữ	Mosquitto, Node.js, MySQL, Ollama	Tiếp nhận, lưu lịch sử, xử lý yêu cầu
Ứng dụng	React, REST, WebSocket	Hiển thị và tương tác với người dùng

ESP32 và nền tảng ESPHome

ESP32 là vi điều khiển trung tâm của mô hình. Các chân GPIO cho phép tiếp nhận tín hiệu số hoặc xuất mức điều khiển đến mạch ngoài. Việc chọn chân phải xét chức năng của từng nhóm chân; riêng GPIO34 thuộc nhóm chân chỉ có khả năng nhập trên ESP32 và không có điện trở kéo lên, kéo xuống nội tích hợp [1]. Trong cấu hình của đề tài, chân này được dùng để đọc ngõ ra module ánh sáng.

ESPHome cho phép khai báo thiết bị, cảm biến, công tắc, kết nối mạng và hành động tự động bằng YAML. Cấu hình nhóm sử dụng biến thể esp32 và framework esp-idf, khai báo DHT11 cùng các binary_sensor, switch. Topic trạng thái và topic lệnh được cấu hình rõ ràng để phần mềm máy chủ có thể dùng một giao ước thống nhất [2].

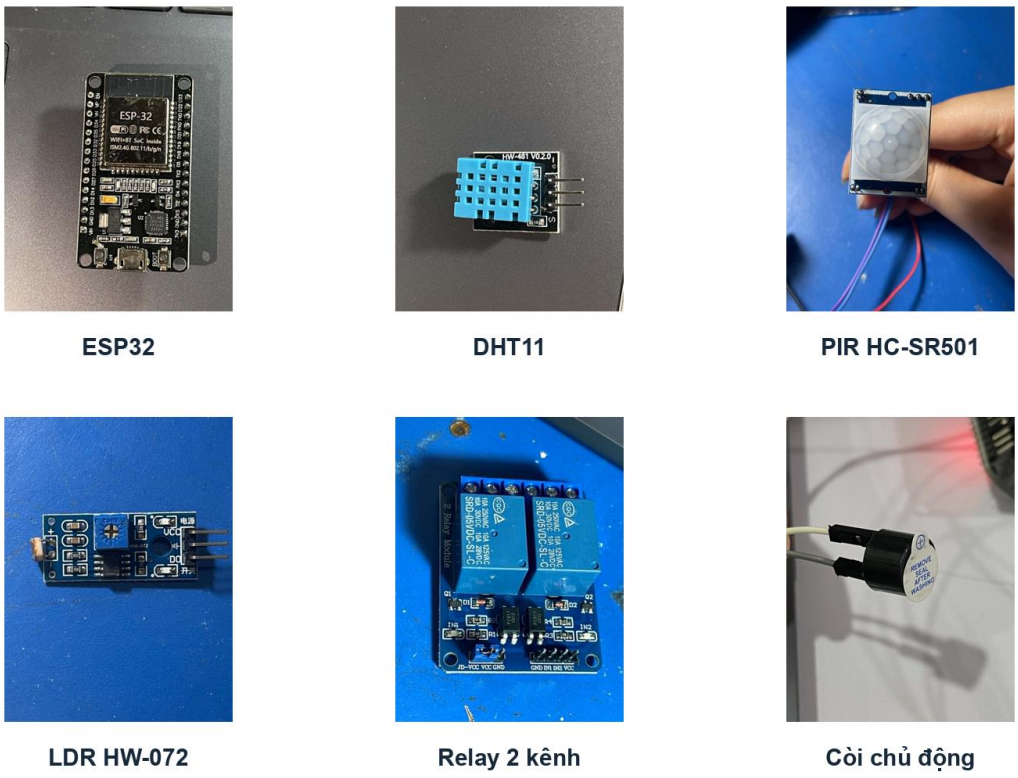
Một ưu điểm của cách triển khai này là logic bật LED và còi nằm trực tiếp trên ESP32. Khi hệ thống đã khởi động và vẫn được cấp nguồn, các nhánh xử lý PIR không cần gửi dữ liệu lên máy chủ rồi chờ lệnh quay về. Tuy vậy, giám sát trên web và truy vấn lịch sử vẫn phụ thuộc vào các dịch vụ máy chủ và kết nối mạng.

Các cảm biến và cơ cấu chấp hành

DHT11 được dùng để thu nhiệt độ và độ ẩm tương đối. Trong ESPHome, một thành phần dht có thể tạo ra hai kênh sensor với topic trạng thái riêng. Nhóm chỉ định model DHT11 và chân dữ liệu GPIO4. Do cấu hình không khai báo update_interval, chu kỳ mặc định theo tài liệu thành phần DHT là 60 giây [3]. Chu kỳ thực tế trong lịch sử có thể bị ảnh hưởng bởi gián đoạn chạy thử hoặc thời điểm kết nối lại.

PIR HC-SR501 đóng vai trò nguồn sự kiện chuyển động, còn module LDR HW-072 đưa ra trạng thái nhị phân theo ngưỡng của mạch so sánh. Chương trình đọc hai ngõ ra này như binary_sensor. Vì dữ liệu ánh sáng chỉ là ON hoặc OFF, nhóm sử dụng nó để quyết định một điều kiện logic; không diễn giải giá trị này thành một phép đo độ rọi liên tục.

Relay tách tín hiệu điều khiển GPIO khỏi mạch tải qua cơ cấu đóng cắt của module. Trong cấu hình, relay ở GPIO26 được đặt inverted: true để ánh xạ lệnh bật sang mức điện phù hợp với module. LED và còi được khai báo dưới dạng switch GPIO, lần lượt tại GPIO25 và GPIO33. Trạng thái phần mềm phản ánh lệnh xuất GPIO; hệ thống chưa có cảm biến dòng hoặc tiếp điểm phản hồi để chứng minh tải đã hoạt động vật lý.



Hình 2.1 Các thành phần phần cứng sử dụng trong mô hình

WiFi và giao tiếp trong mạng nội bộ

WiFi được chọn vì mô hình triển khai trong phạm vi một phòng và có thể dùng bộ phát mạng sẵn có. ESP32 và máy chủ phải truy cập được broker theo địa chỉ IP trong LAN. Trong cấu hình nhóm cung cấp, broker có địa chỉ 192.168.0.102, cổng 1883; phía backend dùng localhost vì kết nối đến broker được ánh xạ trên cùng máy chủ.

Cần phân biệt địa chỉ dùng trên máy chủ với địa chỉ dùng trên thiết bị khác. localhost trong trình duyệt luôn chỉ đến chính thiết bị mở trình duyệt. Vì dashboard hiện đặt API_BASE là http://localhost:3000 và WS_URL là ws://localhost:8081, việc chạy trên máy chủ là thuận tiện; khi mở từ điện thoại hoặc máy khác, các giá trị này cần trỏ về địa chỉ LAN của máy chủ.

Kiến trúc cục bộ giúp các dịch vụ chính không phải gửi dữ liệu ra một nền tảng đám mây bên ngoài. Tuy nhiên, khả năng hoạt động vẫn phụ thuộc nguồn cấp ESP32, bộ phát WiFi, broker và máy tính chạy ứng dụng. Việc truy cập từ xa cần được thiết kế như một chức năng riêng có kiểm soát truy cập phù hợp.

Giao thức MQTT

MQTT dùng mô hình publish/subscribe: bên gửi công bố bản tin lên topic và broker chuyển bản tin đến các bên đã đăng ký. Cách giao tiếp này phù hợp với các thiết bị có tài nguyên hạn chế và các thông điệp ngắn. MQTT có các mức QoS để lựa chọn cơ chế chuyển giao bản tin [4]. Trong mã nguồn backend của đề tài, subscribe và publish không truyền tùy chọn QoS riêng.

Cây topic của hệ thống bắt đầu bằng nha/phongkhach. Các topic kết thúc bằng /state biểu diễn trạng thái, còn /set biểu diễn lệnh. Backend đăng ký nha/phongkhach/# nên nhận cả hai loại. MQTT hỗ trợ bản tin retained để phía đăng ký mới nhận lại trạng thái đã lưu; tài liệu ESPHome cũng cho phép tùy chỉnh state_topic, command_topic và hành vi retain [2].

Nhóm dùng retained để duy trì giá trị tham khảo khi khởi động lại phần mềm. Tuy nhiên, retained không chứng minh thiết bị đang trực tuyến và không cung cấp thời điểm đo gốc trong payload hiện tại. Vì vậy, phần lưu trữ và giao diện cần phân biệt thời điểm nhận lại bản tin với thời điểm cảm biến thực sự tạo phép đo.

REST API và WebSocket

REST API được tổ chức theo các đường dẫn để lấy lịch sử, gửi lệnh, quản lý thói quen và gọi chatbot. Express định tuyến yêu cầu dựa trên phương thức HTTP và đường dẫn; các tham số trên đường dẫn có thể được sử dụng để xác định loại dữ liệu cần truy vấn [5]. Trong đề tài, GET /api/lichsu/:loai ánh xạ tên loại sang topic cảm biến tương ứng.

WebSocket được dùng cho các cập nhật đẩy từ backend tới dashboard. Khác với việc tải lại toàn bộ trang, dashboard giữ một kết nối và nhận các JSON mà máy chủ phát ra. Mỗi bản tin MQTT được ghi MySQL thành công sẽ được broadcast qua cổng 8081. Việc tách REST và WebSocket giúp sử dụng REST để khởi tạo dữ liệu, còn WebSocket để cập nhật khi có bản tin mới.

Trạng thái kết nối WebSocket chỉ cho biết trình duyệt đã kết nối đến backend. Để đánh giá toàn bộ đường truyền, còn cần biết backend có kết nối broker và ESP32 có hoạt động hay không. Đây là ba lớp trạng thái khác nhau; chỉ hiển thị một nhãn kết nối chung có thể khiến người dùng hiểu sai độ mới của dữ liệu.

Node.js và cơ sở dữ liệu MySQL

Backend viết bằng JavaScript, chạy trên Node.js và kết hợp các thư viện mqtt, express, mysql2/promise, ws, cors và dotenv. Chương trình nhận bản tin, ghi dữ liệu, phát thông báo và xử lý HTTP trong một dịch vụ. Các thao tác truy vấn cơ sở dữ liệu dùng lời gọi bất đồng bộ để chờ kết quả trước khi tiếp tục xử lý.

MySQL lưu trữ lịch sử dưới dạng các hàng có topic, giá trị và thời gian nhận. Cách thiết kế chung một bảng cho nhiều loại bản tin giúp thêm topic mà không phải thêm cột. Đổi lại, trường giá trị là chuỗi nên các truy vấn thống kê nhiệt độ và độ ẩm phải chuyển kiểu số; đồng thời cần chuẩn hóa payload trước khi sử dụng cho các phép tính.

Một bảng riêng lưu danh mục và nội dung thói quen người dùng. Backend đưa dữ liệu này vào ngữ cảnh chatbot. Trong phiên bản hiện tại, đây là thông tin do người dùng ghi lại; hệ thống chưa tự học thói quen từ dữ liệu và chưa có bộ lập lịch biến một câu mô tả thói quen thành lệnh điều khiển tự động.

Giao diện React và trợ lý AI cục bộ

React tổ chức giao diện thành các thành phần như thẻ cảm biến, khối điều khiển relay, biểu đồ, danh mục thiết bị và trang hội thoại. State lưu các giá trị được hiển thị; các effect thực hiện tác vụ tải dữ liệu hoặc thiết lập kết nối khi thành phần được gắn vào trang [6]. Dashboard của đề tài dùng cách tổ chức này để kết hợp dữ liệu REST ban đầu với dữ liệu WebSocket.

Ollama cung cấp API HTTP để gọi mô hình ngôn ngữ chạy cục bộ. API /api/chat nhận model và messages; tùy chọn stream: false yêu cầu trả về một phản hồi hoàn chỉnh thay vì chuỗi kết quả theo thời gian [7]. Backend đang gọi model llama3.1:latest tại cổng 11434 và hướng dẫn mô hình trả lời ngắn bằng tiếng Anh.

Phần điều khiển qua câu lệnh và phần hỏi đáp có vai trò khác nhau. Câu lệnh đèn, còi được nhận diện bằng regex trước khi gọi mô hình; các câu hỏi khác được gửi kèm dữ liệu tham chiếu đến LLM. Kiến trúc hiện tại không huấn luyện lại mô hình, không tự học từ lịch sử hội thoại và không để LLM tự sinh lệnh MQTT để thực thi.

CHƯƠNG 3 PHÂN TÍCH THIẾT KẾ VÀ TRIỂN KHAI

Yêu cầu chức năng

Hệ thống được thiết kế cho người sử dụng trong mạng nội bộ. Người dùng cần quan sát thông tin phòng khách, nhận gợi ý từ dữ liệu, điều khiển một số đầu ra và hỏi trợ lý. Các yêu cầu được chia theo luồng thu thập, điều khiển và khai thác dữ liệu để đối chiếu với các thành phần triển khai.

Bảng 3.1 Các yêu cầu chức năng

Mã	Chức năng	Cách thực hiện
CN01	Giám sát nhiệt độ và độ ẩm	Đọc DHT11, gửi hai topic, hiển thị thẻ và biểu đồ
CN02	Giám sát chuyển động và ánh sáng	Đọc PIR, LDR dưới dạng trạng thái số
CN03	Tự động điều khiển tại thiết bị	Xử lý on_press và on_release của PIR trong ESPHome
CN04	Điều khiển relay từ web	POST /api/relay rồi publish relay/set
CN05	Điều khiển LED và còi bằng câu lệnh	Nhận diện câu lệnh, bật chế độ thủ công và publish lệnh
CN06	Theo dõi chế độ vận hành	Nhận thucong/state; cho phép quay lại tự động
CN07	Lưu và đọc lịch sử	MySQL và GET /api/lichsu/:loai
CN08	Gợi ý và phát hiện bất thường	Ngưỡng cố định và truy vấn dữ liệu lịch sử
CN09	Quản lý thói quen	Thêm, xem, xóa các nội dung theo danh mục
CN10	Hỏi đáp có ngữ cảnh	Đọc dữ liệu, tạo context, gọi Ollama

Yêu cầu vận hành

Hệ thống cần duy trì đường đi của dữ liệu từ cảm biến đến cơ sở dữ liệu, đồng thời thể hiện được trạng thái đầu ra sau thao tác điều khiển. Dashboard phải có cách xử lý mất kết nối và không làm mất dữ liệu lịch sử khi đóng trình duyệt. Trong mã hiện tại, WebSocket được kết nối lại sau 3 giây nếu đường truyền bị đóng.

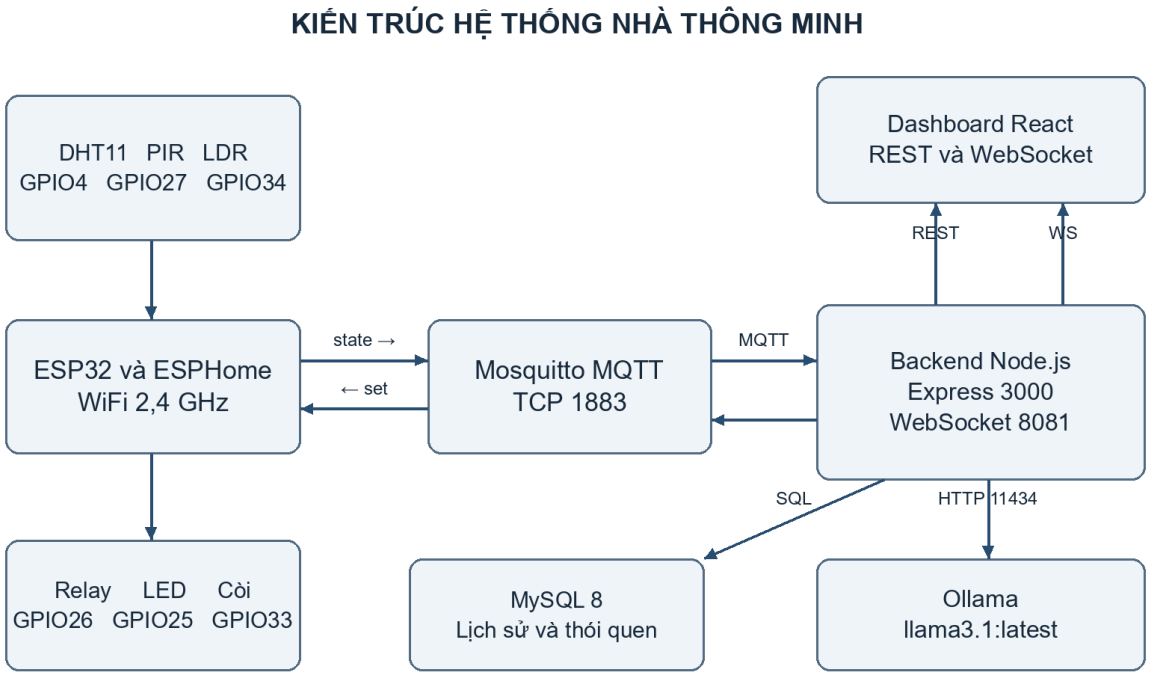
Máy chủ cần chạy đồng thời broker, MySQL, backend và dashboard; Ollama cần sẵn sàng khi sử dụng nhánh hỏi đáp AI. Cổng 1883 dùng cho MQTT, 3000 cho REST

API, 8081 cho WebSocket và 11434 cho Ollama. Cổng 5173 được sử dụng khi chạy Vite phục vụ phát triển giao diện.

Cấu hình xác thực được tách khỏi nội dung công khai của báo cáo. Mật khẩu cơ sở dữ liệu nằm trong biến môi trường của backend; các mật khẩu WiFi, MQTT, OTA và khóa API của ESPHome được biểu diễn bằng tham chiếu secrets trong phụ lục. Phụ lục giữ nguyên logic điều khiển để có thể đối chiếu với phiên bản của nhóm.

Kiến trúc tổng thể

Kiến trúc có hai điểm xử lý chính. ESP32 xử lý các phản ứng theo sự kiện ngay tại mô hình; backend xử lý lưu trữ, giao diện và trợ lý. MQTT broker là điểm trung chuyển hai chiều. Dashboard không trực tiếp truy cập GPIO và không cần kết nối MQTT riêng, mà giao tiếp với backend qua HTTP và WebSocket.



Hình 3.1 Sơ đồ kiến trúc và giao tiếp giữa các khối

Trong chiều giám sát, cảm biến tạo dữ liệu, ESP32 publish các topic trạng thái, broker chuyển đến backend, backend ghi MySQL rồi phát JSON cho trình duyệt. Trong chiều điều khiển, dashboard gửi HTTP, backend kiểm tra yêu cầu rồi publish topic lệnh, ESP32 tiếp nhận và cập nhật trạng thái đầu ra. Topic /state được dùng để phản ánh kết quả ở mức phần mềm.

Mô hình đặt các dịch vụ xử lý trong LAN giúp quá trình thử nghiệm có thể quan sát trực tiếp. Broker được chạy bằng container Mosquitto; MySQL cung cấp dữ liệu lưu bền; Ollama chạy trên máy tính. Không có yêu cầu bắt buộc một dịch vụ cloud

trong đường xử lý chính, nhưng toàn bộ chuỗi giám sát vẫn cần các thành phần cục bộ hoạt động đồng thời.

Thiết kế phần cứng và phân bố chân

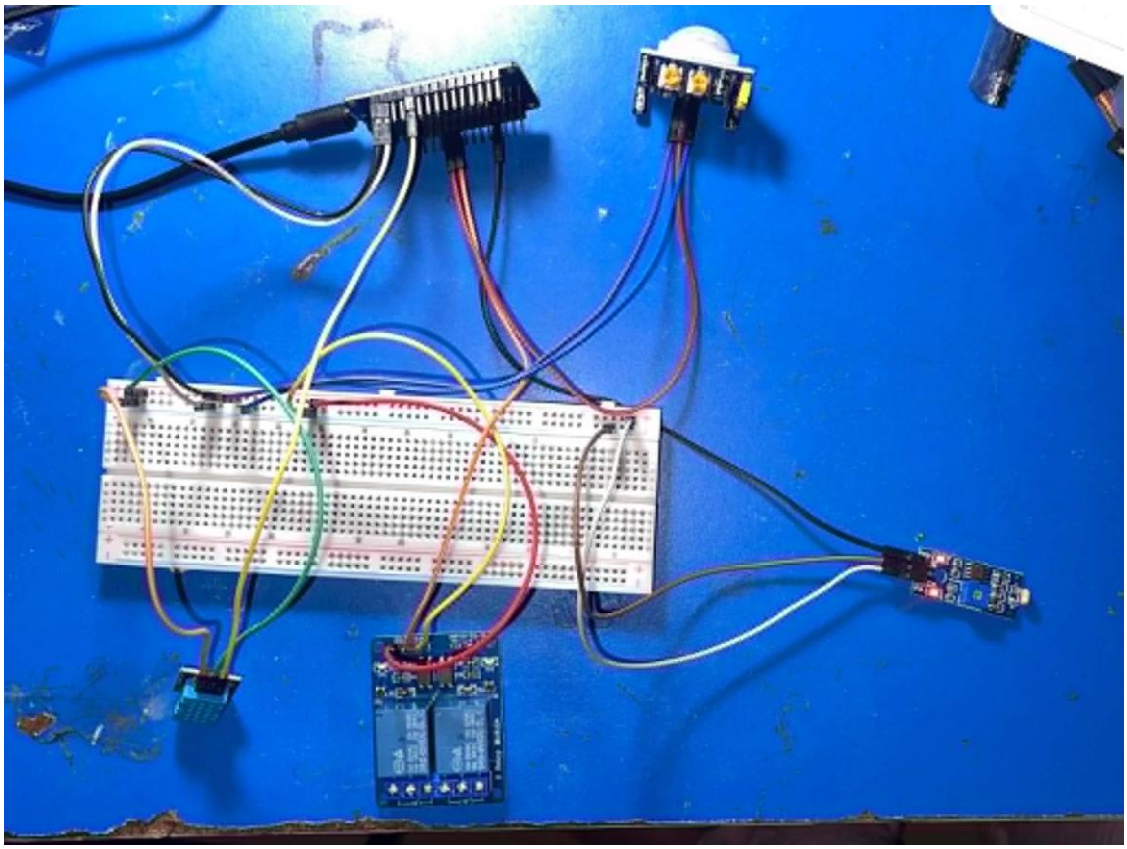
Bảng 3.2 Phân bố chân theo cấu hình ESPHome

Thiết bị	GPIO	Kiểu tín hiệu	Cấu hình quan trọng
DHT11	4	Dữ liệu vào và giao tiếp DHT	model DHT11; hai kênh nhiệt độ, độ ẩm
PIR HC-SR501	27	Ngõ vào số	on_press và on_release kích hoạt tự động
Module LDR	34	Ngõ vào số	Không khai báo inverted; chân chỉ nhập
LED chuyển động	25	Ngõ ra số	switch GPIO; có state_topic và command_topic
Relay phòng khách	26	Ngõ ra số	inverted: true; một kênh được cấu hình
Còi chủ động	33	Ngõ ra số	switch GPIO; còi tự động bật 500 ms

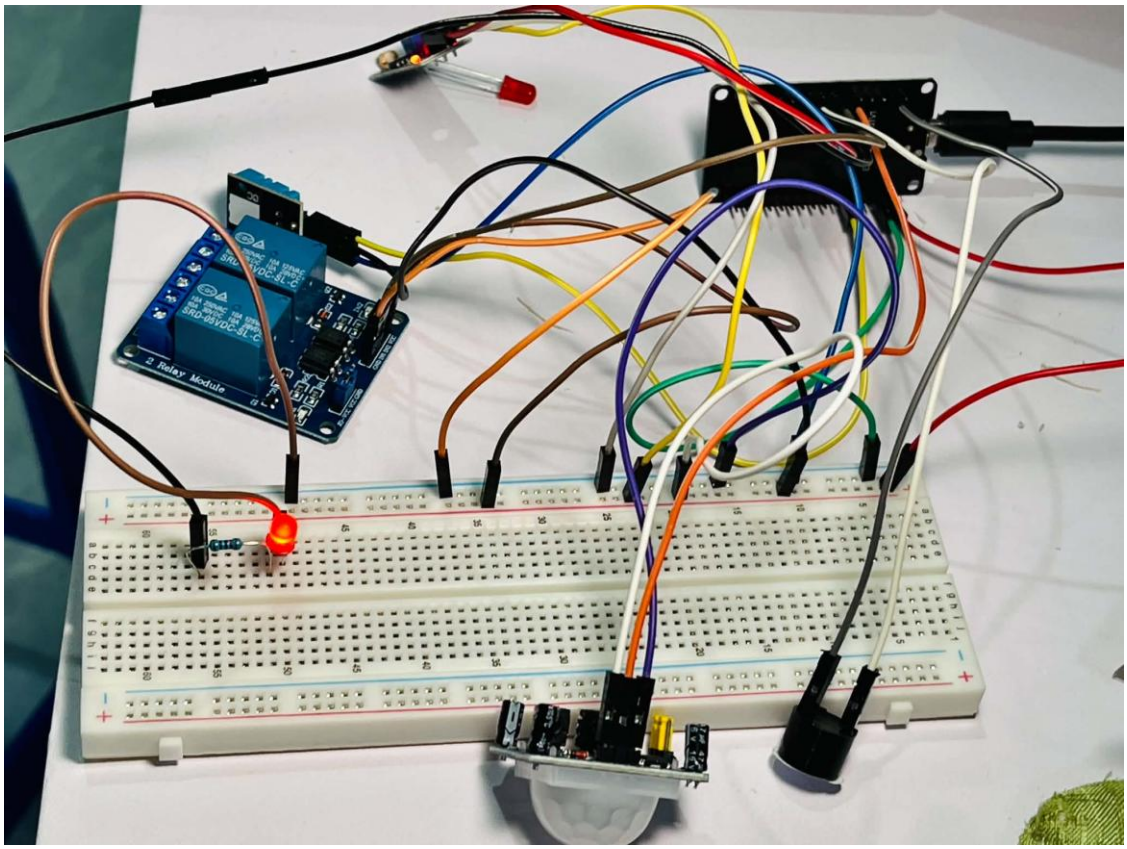
DHT11 sử dụng một chân dữ liệu tại GPIO4 để cung cấp cả nhiệt độ và độ ẩm. Hai cảm biến nhị phân được đấu tới GPIO27 và GPIO34. Các khối có chung tham chiếu GND theo sơ đồ lắp của mô hình. Tín hiệu đưa về ESP32 phải phù hợp với mức logic của vi điều khiển; không suy ra mức điện áp ngõ ra của một module chỉ từ tên gọi nguồn cấp.

LED cần điện trở hạn dòng phù hợp. Với còi và module relay, việc chọn nguồn và tầng kích cần dựa trên dòng tiêu thụ thực tế của linh kiện. GPIO chỉ có vai trò phát tín hiệu điều khiển; khi triển khai với cơ cấu có dòng tải lớn hơn khả năng chân, cần tầng đệm phù hợp. Đối với tải điện lưới, phần tiếp điểm và dây dẫn phải được bố trí tách khỏi mạch tín hiệu thấp áp.

Ảnh mô hình cho thấy nhóm đã triển khai trên breadboard và dùng các dây nối để thuận tiện thay đổi cấu hình. Phiên bản đầu gồm ESP32, DHT11, PIR, LDR và relay. Phiên bản sau bổ sung LED, còi cùng chế độ thủ công. Việc chia phiên bản giúp thử riêng luồng đọc cảm biến trước khi mở rộng các hành động đầu ra.



Hình 3.2 Mô hình lắp ráp phiên bản đầu



Hình 3.3 Mô hình bổ sung LED và còi ở phiên bản sau

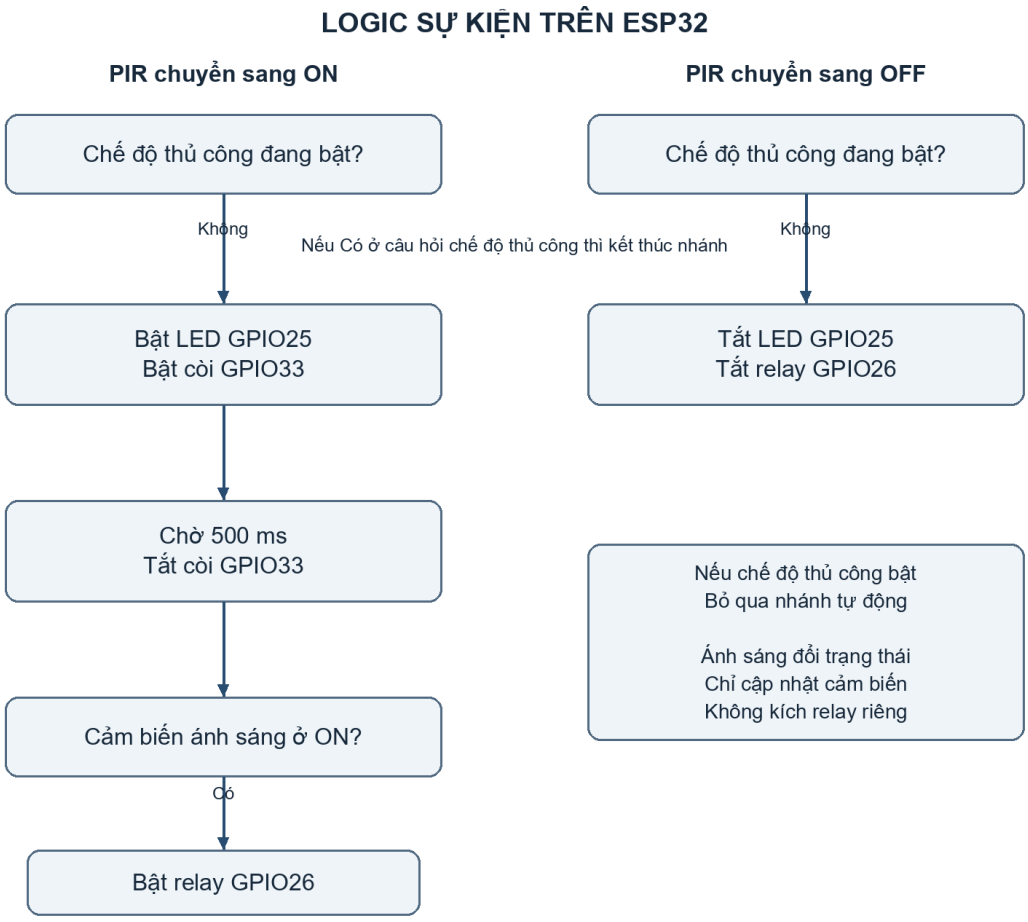
Thiết kế firmware và chế độ hoạt động

Thiết bị có tên cam-bien-phong-khach. Cấu hình khai báo logger, WiFi, MQTT, native API và OTA. MQTT discovery được tắt để sử dụng trực tiếp các topic do nhóm đặt. Một global kiểu bool có id che_do_thu_cong lưu cờ vận hành; restore_value: no và initial_value: false cho biết giá trị khởi tạo của biến này là chế độ tự động.

Công tắc template cong_tac_thu_cong nhận ON hoặc OFF qua thucong/set. Hai hành động turn_on_action và turn_off_action thay đổi global tương ứng. Tùy chọn optimistic giúp công tắc công bố trạng thái theo lệnh được xử lý, không dựa trên một cảm biến đo chế độ bên ngoài.

Ở nhánh on_press của PIR, chương trình chỉ tiếp tục khi cờ thủ công đang tắt. ESP32 bật LED, bật còi, chờ 500 ms rồi tắt còi. Sau bước này, chương trình kiểm tra binary_sensor ánh sáng; nếu trạng thái đang ON thì bật relay. Do vị trí lệnh trong YAML, relay được xét sau khoảng chờ của còi, chứ không được bật ngay ở đầu nhánh.

Ở nhánh on_release, nếu vẫn ở chế độ tự động, ESP32 tắt LED và relay. Cảm biến ánh sáng không có on_press hoặc on_release riêng để kích relay, nên việc ánh sáng thay đổi khi PIR vẫn giữ nguyên không tự tạo một lần đánh giá mới. Khi chuyển từ thủ công về tự động, cấu hình cũng chưa có hành động tính lại toàn bộ đầu ra ngay tại thời điểm chuyển chế độ.



Hình 3.4 Lưu đồ xử lý sự kiện PIR theo cấu hình hiện tại

Bảng 3.3 Hành động tự động theo sự kiện

Sự kiện	**Điều kiện**	**Hành động**
PIR chuyển ON	Thủ công OFF	Bật LED; còi bật 500 ms rồi tắt
Sau nhánh còi	Thủ công OFF và ánh sáng ON	Bật relay
PIR chuyển OFF	Thủ công OFF	Tắt LED và relay
PIR thay đổi	Thủ công ON	Bỏ qua nhánh tự động
Ánh sáng thay đổi	Mọi chế độ	Cập nhật trạng thái; không có nhánh relay riêng
Thủ công chuyển OFF	Lệnh người dùng	Đổi cờ; chờ các sự kiện tiếp theo

Trạng thái ON của cảm biến ánh sáng trong firmware là mức logic dùng trong điều kiện tự động. Dashboard hiện đảo nhãn hiển thị ánh sáng để phù hợp cách nhóm quan sát module, nên cần phân biệt nhãn trên màn hình với giá trị MQTT gốc. Việc

thống nhất cực tính ở firmware và giao diện sẽ giúp quy tắc điều khiển dễ giải thích hơn khi mở rộng mô hình.

Thiết kế cây topic và payload MQTT

Các topic được đặt theo cấu trúc phạm vi nhà, phòng, loại dữ liệu và hướng giao tiếp. Nhóm dùng chung tiền tố nha/phongkhach để backend có thể subscribe một nhánh. Topic cụ thể giúp tách riêng các kênh mà không cần đóng gói tất cả cảm biến vào một payload lớn.

Bảng 3.4 Các topic trạng thái và lệnh

Nhánh sau nha/phongkhach/	Payload hiện tại	Hướng và vai trò
nhietdo/state	Chuỗi số theo °C	ESP32 gửi nhiệt độ
doam/state	Chuỗi số theo %	ESP32 gửi độ ẩm tương đối
chuyendong/state	ON hoặc OFF	ESP32 gửi trạng thái PIR
anhsang/state	ON hoặc OFF	ESP32 gửi mức logic ánh sáng
relay/state và relay/set	ON hoặc OFF	Phản hồi relay và lệnh relay
den/state và den/set	ON hoặc OFF	Phản hồi LED và lệnh LED
coi/state và coi/set	ON hoặc OFF	Phản hồi còi và lệnh còi
thucong/state và thucong/set	ON hoặc OFF	Trạng thái và lệnh chọn chế độ

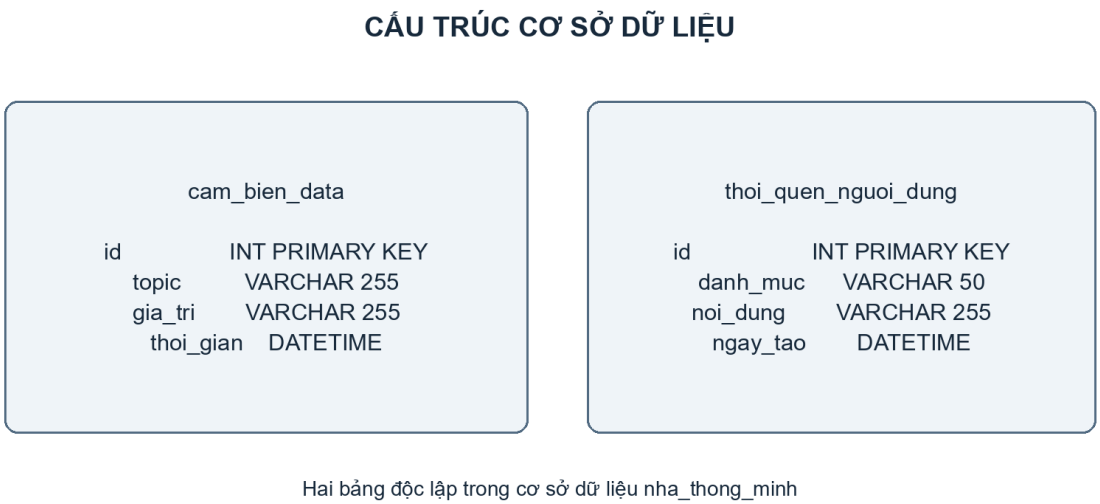
Trong lịch sử còn tồn tại một số payload JSON tại den/state từ phiên bản khai báo LED trước đó. Với cấu hình switch hiện tại, payload dự kiến là ON hoặc OFF. Vì bảng dữ liệu lưu nguyên chuỗi nhận được, phần phân tích cần chuẩn hóa theo từng phiên bản thay vì coi mọi giá trị trong cùng topic có cùng định dạng.

Backend đang ghi mọi topic dưới nha/phongkhach/# vào cùng bảng cam_bien_data, bao gồm cả lệnh /set. Nhờ đó có thể đối chiếu một số lần gửi lệnh với phản hồi, nhưng tổng số hàng không bằng tổng số phép đo cảm biến. Việc tách một bảng lệnh riêng hoặc bổ sung loại bản tin sẽ giúp các truy vấn thống kê rõ nghĩa hơn.

Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu nha_thong_minh có hai bảng: cam_bien_data và thoi_quen_nguoi_dung. Khóa chính id kiểu INT tự tăng nhận diện từng hàng. Hai bảng độc lập và không có quan hệ khóa ngoại trong cấu trúc hiện tại. Thời gian ở các

trường DATETIME được tạo theo mặc định CURRENT_TIMESTAMP của MySQL khi thêm hàng.



Hình 3.5 Cấu trúc hai bảng dữ liệu chính

Bảng 3.5 Các trường trong bảng cam_bien_data

Trường	Kiểu	Ràng buộc và ý nghĩa
id	INT	Khóa chính, AUTO_INCREMENT
topic	VARCHAR 255	NOT NULL; topic bản tin nhận được
gia_tri	VARCHAR 255	NOT NULL; payload dạng chuỗi
thoi_gian	DATETIME	Mặc định CURRENT_TIMESTAMP; thời điểm ghi nhận

Bảng 3.6 Các trường trong bảng thoi_quen_nguoi_dung

Trường	Kiểu	Ràng buộc và ý nghĩa
id	INT	Khóa chính, AUTO_INCREMENT
danh_muc	VARCHAR 50	NOT NULL; nhóm nội dung
noi_dung	VARCHAR 255	NOT NULL; mô tả thói quen
ngay_tao	DATETIME	Mặc định CURRENT_TIMESTAMP

Thời gian MySQL trong lần kiểm tra sử dụng SYSTEM với múi giờ hệ điều hành SE Asia Standard Time. Khi xuất số liệu, nhóm đọc DATETIME ở dạng chuỗi để giữ nguyên giờ địa phương và tránh cộng múi giờ hai lần. Trên dashboard, các thời gian từ

REST và thời gian tạo tại trình duyệt cần được thống nhất định dạng nếu muốn ghép dữ liệu theo cùng một trục thời gian.

Cấu trúc bảng hiện có khóa chính nhưng chưa có chỉ mục ghép theo topic và thoi_gian. Với số lượng bản ghi nhỏ, hệ thống vẫn truy vấn được lịch sử. Khi dữ liệu tăng, chỉ mục này có thể giúp các truy vấn WHERE topic và ORDER BY thoi_gian làm việc hiệu quả hơn; đồng thời cần quy định thời hạn lưu và cơ chế sao lưu dữ liệu.

Xử lý dữ liệu tại backend

Backend tạo một MQTT client kết nối mqtt://localhost:1883 và đăng ký nha/phongkhach/# sau khi kết nối thành công. Mỗi lần nhận dữ liệu, chương trình chuyển buffer thành chuỗi, thực hiện INSERT có tham số rồi gọi broadcast với JSON gồm topic và gia_tri. Dữ liệu được lưu trước khi gửi đến các WebSocket client đang ở trạng thái OPEN.

Cách dùng placeholder trong truy vấn INSERT giúp truyền payload qua tham số thay vì ghép trực tiếp vào chuỗi SQL. Hàm broadcast duyệt các kết nối hiện có và gửi cùng một JSON đến từng client còn mở. Dashboard ánh xạ topic sang khóa state tương ứng để cập nhật các thẻ và chuỗi điểm dữ liệu.

Do việc broadcast đặt sau câu lệnh await ghi dữ liệu, lỗi MySQL có thể làm gián đoạn nhánh cập nhật giao diện. Backend hiện cần bổ sung xử lý lỗi riêng cho luồng MQTT và thông báo trạng thái dịch vụ để phản ánh tình huống cơ sở dữ liệu tạm thời không sẵn sàng. Đây là cải tiến về độ ổn định khi chuyển từ mô hình thử nghiệm sang vận hành dài ngày.

Thiết kế các REST API

Backend sử dụng cổng 3000, cho phép parse JSON và nhận các yêu cầu từ dashboard. Các API truy vấn trả JSON để giao diện có thể dùng trực tiếp. Với lịch sử, đường dẫn :loai được ánh xạ qua danh sách định sẵn để tránh chọn topic tùy ý từ đầu vào người dùng.

Bảng 3.7 Các API của hệ thống

Phương thức và đường dẫn	Đầu vào chính	Kết quả
GET /api/lichsu/:loai	nhietdo, doam, chuyendong, anhsang, relay	Tối đa 50 hàng mới nhất, gồm gia_tri và thoi_gian
POST /api/relay	trang_thai: ON hoặc OFF	Publish relay/set; trả thông báo đã gửi lệnh
POST /api/che-do-thu-cong	trang_thai: ON hoặc OFF	Publish thucong/set
GET /api/bat-thuong	Không có	Danh sách mô tả bất thường và ghi chú nếu chưa đủ thời gian
GET /api/thoi-quen	Không có	Danh sách nhóm theo danh_muc
POST /api/thoi-quen	danh_muc, noi_dung	Thêm hàng và trả id, danh mục, nội dung
DELETE /api/thoi-quen/:id	Mã bản ghi	Xóa mục thói quen theo id
POST /api/chatbot	cau_hoi	tra_loi hoặc thông báo lỗi

Khi nhận lệnh relay hoặc chế độ thủ công, backend kiểm tra giá trị chỉ được là ON hoặc OFF; đầu vào khác trả HTTP 400. Với loại lịch sử không có trong ánh xạ, backend trả HTTP 404. API thêm thói quen kiểm tra không được để trống và giới hạn độ dài tên danh mục là 50 ký tự.

Phản hồi HTTP thành công của API điều khiển chỉ xác nhận backend đã xử lý yêu cầu gửi lệnh. Chương trình không chờ một thông điệp xác nhận riêng có mã giao dịch từ ESP32. Vì thế, việc hiển thị kết quả cần dựa thêm vào topic trạng thái; để xác minh tác động vật lý còn cần phản hồi từ mạch đo hoặc quan sát tại mô hình.

Gợi ý môi trường và phát hiện bất thường

Hàm kiemTraGoiY áp dụng các điều kiện cố định cho nhiệt độ và độ ẩm. Mỗi khi nhận bản tin, backend tạo gợi ý và so sánh nội dung với lần trước trên cùng topic. Nếu nội dung thay đổi, chương trình phát JSON type: canh_bao qua WebSocket. Cơ chế này giảm việc gửi lặp cùng một câu, nhưng khi giá trị số thay đổi, nội dung câu cũng thay đổi dù vẫn thuộc một mức cảnh báo.

Bảng 3.8 Các ngưỡng gợi ý trong mã nguồn

Đại lượng	Điều kiện	Phản hồi
Nhiệt độ	T > 33°C	canh_bao; gợi ý quạt hoặc điều hòa
Nhiệt độ	30°C < T ≤ 33°C	goi_y; cân nhắc bật quạt
Nhiệt độ	T ≤ 30°C	Không tạo gợi ý nhiệt độ
Độ ẩm	H < 40%	goi_y; độ ẩm thấp
Độ ẩm	H > 80%	goi_y; độ ẩm cao
Độ ẩm	40% ≤ H ≤ 80%	Không tạo gợi ý độ ẩm

API bất thường dùng thời gian từ bản ghi đầu tiên đến hiện tại để quyết định có đủ tối thiểu 3 ngày hay chưa. Sau đó, chương trình tính nhiệt độ trung bình của các hàng trong 7 ngày gần nhất và so với nhiệt độ mới nhất; nếu độ lệch tuyệt đối lớn hơn 3°C thì thêm thông báo. Với chuyển động, chương trình tìm xem trong khung giờ hiện tại có bản tin ON hôm nay nhưng không có bản tin ON nào ở cùng giờ trong các ngày trước trong cửa sổ 7 ngày hay không.

Đây là các quy tắc thống kê đơn giản, không phải mô hình học máy được huấn luyện. Điều kiện đủ 3 ngày hiện tại đo khoảng cách từ bản ghi đầu đến thời điểm chạy, không kiểm tra có đủ 3 ngày dữ liệu thực sự. Khi dữ liệu không liên tục hoặc bị phát lại, kết quả cần được hiểu theo đặc điểm tập dữ liệu đang có. Nhóm chưa dùng đầu ra này để tự động can thiệp vào thiết bị.

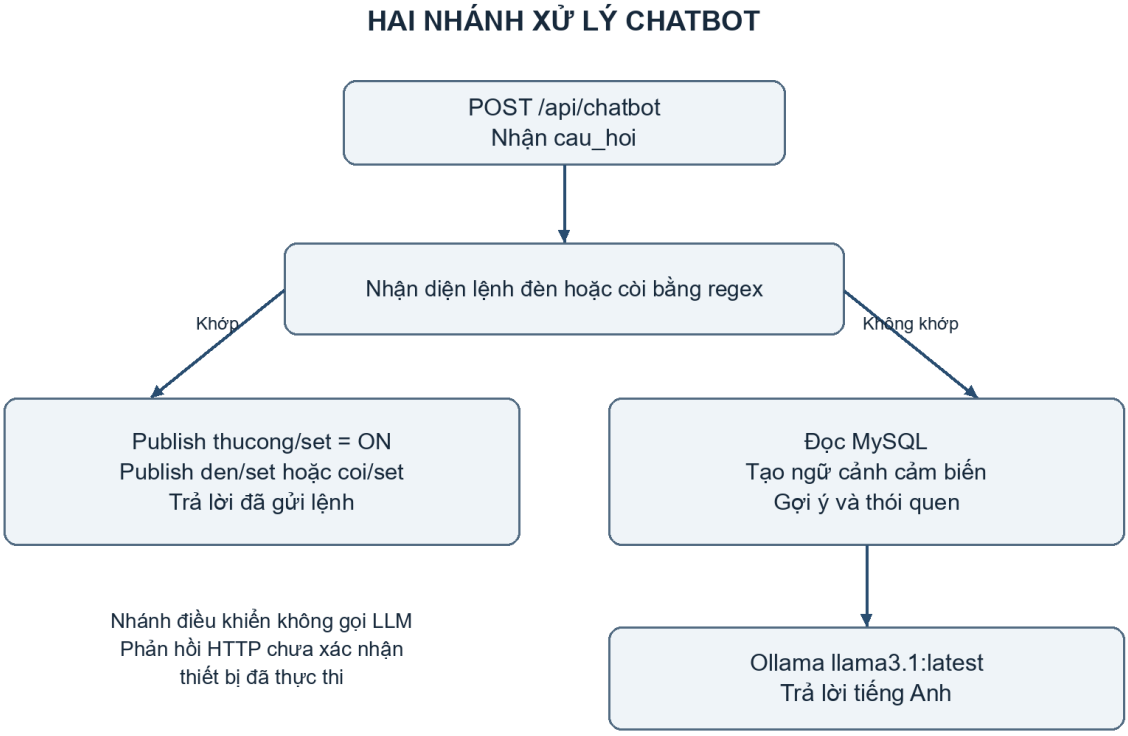
Thiết kế trợ lý AI

Khi nhận cau_hoi, backend gọi phatHienLenh trước. Hàm chuyển chuỗi về chữ thường rồi kiểm tra các mẫu tiếng Việt và tiếng Anh cho bốn thao tác: bật đèn, tắt đèn, bật còi và tắt còi. Nếu khớp, backend publish thucong/set bằng ON, sau đó publish lệnh tới den/set hoặc coi/set và trả lời ngay bằng một câu xác nhận đã thực hiện nhánh điều khiển.

Trong ngữ cảnh này, từ “đèn” trong nhánh chatbot được ánh xạ tới LED GPIO25, không phải relay GPIO26. Relay có API và nút điều khiển riêng. Cách ánh xạ này cần được trình bày rõ cho người dùng để tránh nhầm giữa đèn chỉ báo trên mô hình với một thiết bị chiếu sáng nối qua relay.

Với câu hỏi không khớp lệnh, hàm taoTomTat lấy nhiệt độ, độ ẩm mới nhất; tạo gợi ý; kiểm tra bất thường; đọc thói quen. Backend ghép những nội dung này vào

system message và gọi Ollama. Mỗi yêu cầu chỉ gửi system message và câu hỏi hiện tại; các tin nhắn cũ đang hiển thị trên giao diện không được chuyển thành lịch sử hội thoại đầy đủ cho mô hình.



Hình 3.6 Luồng nhận diện lệnh và hỏi đáp của chatbot

Mô hình được yêu cầu trả lời tiếng Anh trong 2–4 câu ngắn. Nhóm có thể đặt câu hỏi về dữ liệu cảm biến, các gợi ý, bất thường hoặc nội dung thói quen đã ghi. Chất lượng câu trả lời phụ thuộc vào ngữ cảnh được tạo và trạng thái dịch vụ Ollama; dữ liệu cảm biến được giữ trong ngữ cảnh nhưng mô hình không tự kiểm chứng tuổi của bản tin.

Regex giúp triển khai nhanh các câu lệnh đã xác định. Khi câu chứa phủ định, nhiều thiết bị hoặc nhiều hành động, một mẫu đơn giản có thể nhận diện không như ý định. Hướng hoàn thiện là chuẩn hóa tập lệnh, kiểm tra đầu vào và hiển thị rõ trạng thái đã gửi lệnh, đã nhận phản hồi thiết bị, thay vì dùng một câu khẳng định chung cho tất cả các bước.

Thiết kế mạng và phân bố các dịch vụ

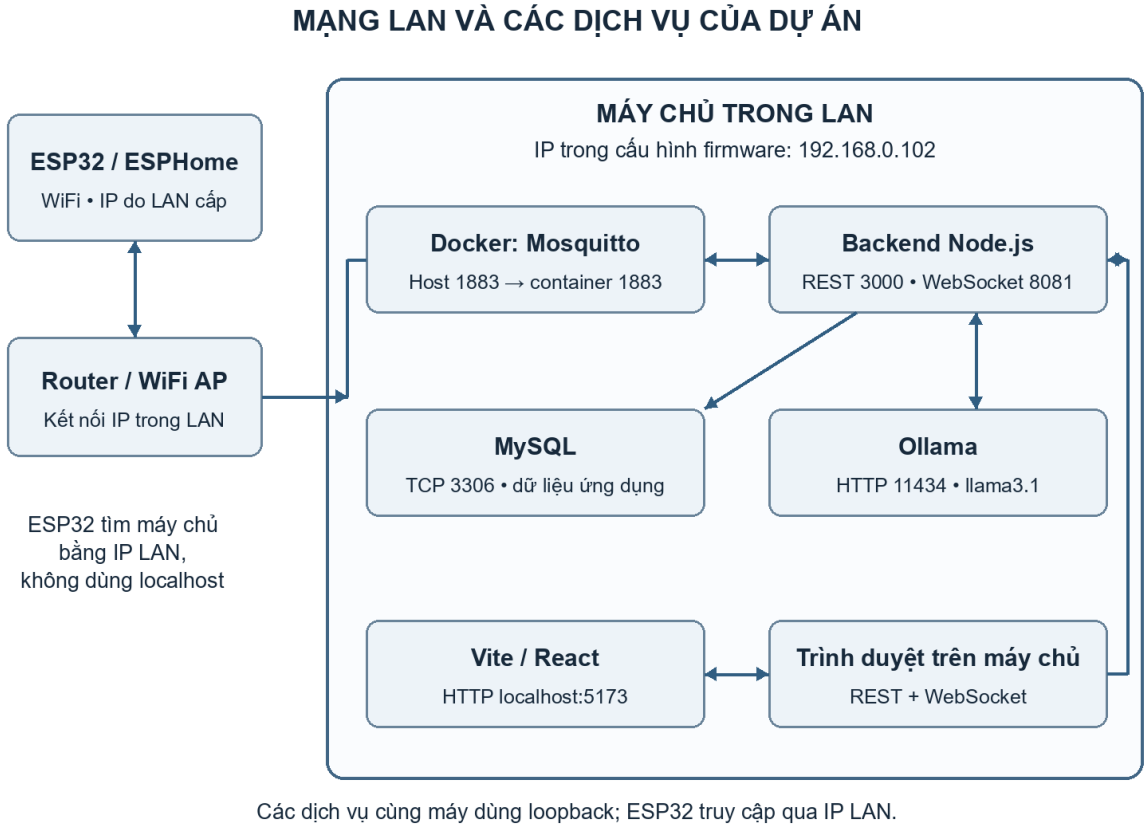
Mô hình kết nối trong mạng nội bộ

Hệ thống sử dụng mạng LAN làm môi trường kết nối giữa nút ESP32 và máy tính chạy các dịch vụ. ESP32 kết nối với điểm truy cập WiFi; máy tính có thể nối với cùng

mạng bằng WiFi hoặc Ethernet. Bộ định tuyến cung cấp kết nối IP giữa hai phía. Vì ESP32 chủ động tạo phiên TCP đến broker, việc tiếp nhận dữ liệu không yêu cầu trình duyệt kết nối trực tiếp đến vi điều khiển.

Ở phía máy tính, nhóm tách các vai trò thành broker bản tin, máy chủ ứng dụng, cơ sở dữ liệu, dịch vụ mô hình AI và máy chủ giao diện. Các thành phần này có thể cùng chạy trên một máy nhưng đảm nhiệm những nhiệm vụ khác nhau. Broker phân phối bản tin; backend quyết định cách lưu, truy vấn và chuyển dữ liệu; MySQL giữ lịch sử; Ollama sinh câu trả lời; React trình bày thông tin cho người dùng. Cách tổ chức này giúp xác định rõ một yêu cầu đi qua những dịch vụ nào khi kiểm tra hệ thống.

Trong cấu hình ESPHome do nhóm sử dụng, địa chỉ broker là 192.168.0.102 và cổng là 1883. Đây là địa chỉ của máy chủ nhìn từ ESP32 trong cấu hình triển khai đó. Khi chuyển sang mạng khác, cần kiểm tra lại địa chỉ IPv4 của máy chủ, dải mạng, kết nối WiFi và quy tắc tường lửa. Ảnh khám phá thiết bị ở một phiên khác có thể chứa địa chỉ thuộc dải 192.168.1.x; hai địa chỉ này thuộc những thời điểm cấu hình khác nhau, không được mặc định là cùng một mạng đang hoạt động.



Hình 3.7 Sơ đồ mạng LAN và vị trí các dịch vụ trên máy chủ

Luồng vận hành chính nằm trong LAN. Sau khi đã cài các gói phần mềm và tải mô hình AI, các bước chuyển trạng thái MQTT, lưu MySQL, truy vấn REST, cập nhật WebSocket và hỏi đáp Ollama đều sử dụng dịch vụ cục bộ. Internet phục vụ việc cài đặt, cập nhật hoặc tải tài nguyên khi cần. Khả năng giám sát vẫn phụ thuộc vào nguồn cấp thiết bị, điểm truy cập và máy chủ hoạt động ổn định.

Phân biệt địa chỉ thiết bị, máy chủ và loopback

Địa chỉ localhost được hiểu theo máy đang thực hiện kết nối. Backend dùng mqtt://localhost:1883 vì backend và cổng broker được công bố trên cùng máy tính. ESP32 không thể dùng localhost để tìm máy tính: trên ESP32, tên đó chỉ chính thiết bị. Do vậy, firmware phải sử dụng IP LAN của máy chủ hoặc một tên máy được mạng nội bộ phân giải đúng.

Tương tự, localhost trong mã frontend được trình duyệt diễn giải trên máy của người xem. Với cách trình diễn hiện tại, trình duyệt mở trên chính máy chủ nên các địa chỉ http://localhost:3000 và ws://localhost:8081 hoạt động phù hợp. Nếu mở dashboard trên điện thoại, các địa chỉ này sẽ trỏ về điện thoại. Chỉ thay URL của trang web sang IP máy tính sẽ chưa đủ; địa chỉ API và WebSocket trong ứng dụng cũng phải được cấu hình theo máy chủ thực tế.

Bảng 3.9 Phân bố dịch vụ, cổng và phía sử dụng

Dịch vụ	Địa chỉ hoặc cổng	Phía kết nối và vai trò
Mosquitto MQTT	TCP 1883; IP LAN của máy chủ hoặc localhost	ESP32 dùng IP LAN; backend và MQTT Explorer trên máy chủ dùng localhost/127.0.0.1.
Backend REST	HTTP 3000	Trình duyệt gọi lịch sử, thói quen, chatbot và gửi yêu cầu điều khiển.
Backend WebSocket	WS 8081	Trình duyệt duy trì kết nối để nhận trạng thái và thông báo mới.
MySQL	TCP 3306; DB_HOST=localhost	Backend kết nối cơ sở dữ liệu nha_thong_minh; trình duyệt không truy cập MySQL trực tiếp.
Ollama	HTTP 127.0.0.1:11434	Backend gọi /api/chat cho nhánh hỏi đáp tự do bằng llama3.1:latest.
Vite / React	HTTP localhost:5173	Máy chủ giao diện dùng trong giai đoạn phát triển và trình diễn.
ESPHome Device Builder	HTTP localhost:6052	Công cụ soạn cấu hình, biên dịch và quản lý firmware; tách khỏi đường dữ liệu của dashboard.

Các cổng ở bảng trên là cổng dịch vụ phía máy chủ; máy khách sử dụng cổng nguồn do hệ điều hành cấp khi mở kết nối. Khi vận hành trong LAN, cần giữ ổn định

IP máy chủ, chẳng hạn bằng cách đặt trước địa chỉ DHCP tại router. Các mật khẩu WiFi, OTA, khóa API và thông tin cơ sở dữ liệu được lưu riêng; bảng địa chỉ chỉ mô tả vị trí kết nối, không chứa thông tin xác thực.

Broker Mosquitto, ánh xạ cổng và lưu bền

Mosquitto được triển khai trong container có tên mosquitto. Kiểm tra cấu hình cho thấy cổng TCP 1883 của container được công bố thành cổng 1883 trên máy chủ. Đường đi của bản tin từ ESP32 là: IP LAN của máy tính → cổng 1883 được Docker công bố → listener 1883 trong container. Vì vậy, firmware không cần biết địa chỉ IP riêng bên trong container.

Tệp cấu hình trên máy chủ được gắn vào /mosquitto/config/mosquitto.conf. Dữ liệu broker được đặt trong volume mosquitto_data tại /mosquitto/data; log được đặt trong volume mosquitto_log tại /mosquitto/log. Cấu hình persistence true cùng persistence_location cho phép Mosquitto lưu trạng thái bền theo cơ chế của broker. Kho này phục vụ hoạt động MQTT; lịch sử ứng dụng vẫn được lưu riêng trong MySQL.

Cấu hình đang sử dụng gồm listener 1883, allow_anonymous true, persistence true và đích log dạng tệp. Docker công bố cổng trên các địa chỉ host 0.0.0.0 và ::; phạm vi truy cập từ máy khác còn phụ thuộc tường lửa và mạng của máy chủ. Ánh xạ cổng cho phép thiết bị trong LAN tiếp cận broker nhưng không tự tạo phân quyền cho từng topic. Khi triển khai rộng hơn mô hình học tập, nhóm cần bổ sung tài khoản, ACL và kiểm soát phạm vi truy cập [11].

Tổ chức trao đổi MQTT và ý nghĩa trạng thái phản hồi

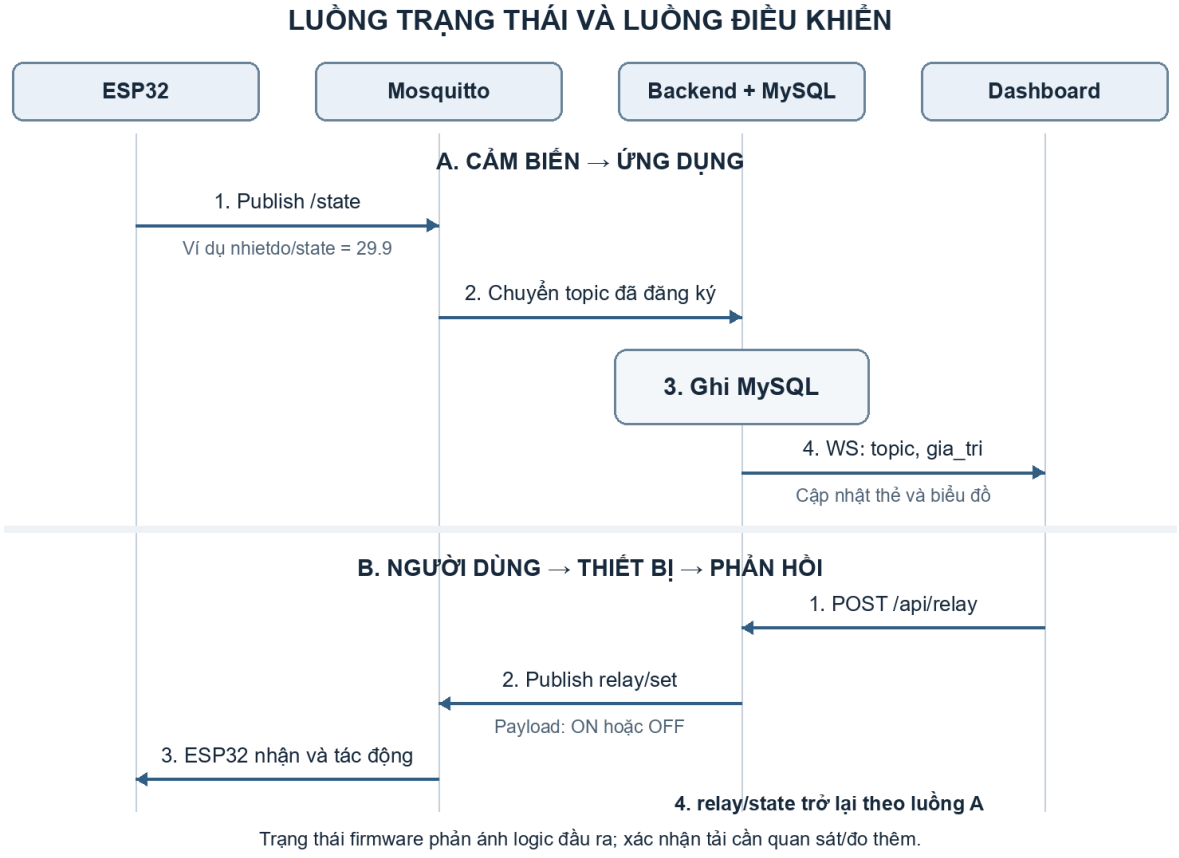
Broker đóng vai trò trung gian giữa bên phát và bên đăng ký nhận. ESP32 công bố các topic kết thúc bằng /state và nhận các topic điều khiển kết thúc bằng /set. Backend đăng ký nha/phongkhach/# để nhận toàn bộ nhánh phòng khách. MQTT Explorer là một client quan sát độc lập, thuận tiện đối chiếu topic và payload mà không phụ thuộc giao diện React.

Trong backend, mỗi bản tin nhận được được chuyển thành chuỗi, ghi vào cam_bien_data rồi mới phát qua WebSocket. Do bộ lọc dùng ký tự #, cả bản tin trạng thái lẫn bản tin lệnh trong nhánh phòng khách đều đi vào bảng. Các API lịch sử lại lọc đúng topic /state của từng loại để hiển thị dữ liệu cảm biến. Sự phân biệt này giúp giải thích vì sao tổng số hàng trong cơ sở dữ liệu lớn hơn số mẫu đo môi trường.

MQTT có ba mức QoS: 0 truyền tối đa một lần; 1 bảo đảm ít nhất một lần và có thể lặp; 2 bảo đảm đúng một lần ở tầng giao thức [10]. Các bản tin retained quan sát khi kiểm tra broker có QoS 0. Dự án chưa cấu hình thêm mã lệnh hoặc cơ chế xác nhận nghiệp vụ để đo tỷ lệ thực thi đầu ra.

Bản tin retained cho phép client mới nhận trạng thái cuối đã lưu trên một topic [10]. Khi backend kết nối lại, giá trị này có thể được ghi thêm với thời điểm nhận mới, dù không phải phép đo mới tại cảm biến. Trong dữ liệu kiểm tra đã gặp 8 trạng thái retained. Do payload hiện không chứa thời điểm đo gốc, cần phân biệt thời gian nhận tại backend với thời gian thiết bị thực sự lấy mẫu.

Ví dụ, POST /api/relay với trang_thai=ON làm backend công bố nha/phongkhach/relay/set. Phản hồi HTTP cho biết yêu cầu đã được xử lý đến bước gửi lệnh. Tiếp đó, ESP32 nhận lệnh, tác động GPIO26 và công bố relay/state. Dashboard chỉ đổi trạng thái hiển thị khi nhận phản hồi trạng thái. Đây là phản hồi logic của firmware; muốn xác nhận tải điện thực sự hoạt động cần thêm tín hiệu đo hoặc quan sát phần cứng. Các lệnh /set hiện được backend gửi mà không đặt retain=true, tránh việc dùng trạng thái lệnh lưu bền như một cơ chế tự chạy lại khi thiết bị kết nối sau.



Hình 3.8 Hai luồng trao đổi trạng thái cảm biến và lệnh điều khiển

REST, WebSocket và ranh giới giữa các dịch vụ

REST phục vụ những thao tác có yêu cầu và phản hồi rõ ràng: lấy 50 hàng lịch sử, đọc hoặc sửa thói quen, gửi yêu cầu bật tắt và hỏi chatbot. WebSocket phục vụ các sự kiện phát sinh sau khi trang đã mở. Nhờ kết hợp hai cơ chế, người dùng thấy dữ liệu đã lưu ngay khi tải trang và tiếp tục nhận thay đổi mà không phải liên tục làm mới trang.

Thông điệp cảm biến trên WebSocket có dạng {"topic":"nha/phongkhach/nhietdo/state","gia_tri":"29.9"}. Frontend ánh xạ topic sang thẻ tương ứng và cập nhật biểu đồ với các đại lượng nhiệt độ, độ ẩm. Thông điệp gợi ý có dạng {"type":"canh_bao","message":"...","muc":"goi_y"}; frontend dùng type và muc để chọn khu vực và màu hiển thị. Một kết nối WebSocket có thể chuyển nhiều loại sự kiện mà không cần mở kết nối riêng cho từng cảm biến.

Frontend và API dùng hai cổng khác nhau nên thuộc hai origin khác nhau. Backend bật middleware cors() để cho phép trình duyệt gọi API. CORS chỉ điều khiển quyền đọc phản hồi trong trình duyệt, không thay thế xác thực người dùng. Nhãn Đã kết nối trên dashboard được đặt khi WebSocket mở; nhãn này không đồng thời chứng minh ESP32 đang trực tuyến, MySQL ghi thành công hoặc Ollama đã sẵn sàng. Mỗi dịch vụ cần được kiểm tra theo đúng vai trò của nó.

Tại thời điểm kiểm tra, Vite lắng nghe trên 127.0.0.1:5173 và Ollama trên 127.0.0.1:11434. Đây là phạm vi truy cập cục bộ. Vite có tùy chọn host cho nhu cầu thử từ thiết bị trong LAN [12]; khi áp dụng còn phải cấu hình lại địa chỉ API, WebSocket và tường lửa. Phiên bản trình diễn dùng HTTP/WS; triển khai lâu dài có thể đặt frontend và API sau một reverse proxy với HTTPS/WSS và cơ chế đăng nhập.

Dịch vụ AI và các thành phần quản trị thiết bị

Ollama là một dịch vụ HTTP riêng, không phải thư viện nhúng trong tiến trình Node.js. Với câu hỏi thông thường, backend truy vấn MySQL, tạo ngữ cảnh gồm nhiệt độ, độ ẩm, gợi ý, bất thường và thói quen rồi gọi http://localhost:11434/api/chat. Yêu cầu chỉ rõ model llama3.1:latest và stream=false; vì vậy backend chờ một kết quả hoàn chỉnh trước khi trả tra_loi cho trình duyệt. Lần gọi đầu có thể bao gồm thời gian nạp mô hình vào bộ nhớ, cần tách khỏi thời gian suy luận khi đánh giá hiệu năng.

Nếu Ollama dừng, các chức năng MQTT, truy vấn lịch sử và relay vẫn có đường xử lý riêng trong backend. Nhánh nhận diện lệnh đèn hoặc còi bằng regex cũng không gọi Ollama. Ngược lại, nhánh hỏi đáp dữ liệu phụ thuộc cả MySQL lẫn dịch vụ mô

hình. Đây là cơ sở để xác định nguyên nhân khi một trang vẫn hiển thị cảm biến nhưng câu hỏi AI không trả lời.

ESPHome Device Builder phục vụ quá trình xây dựng firmware. Native API có khóa mã hóa và OTA có mật khẩu trong cấu hình thiết bị, song đường giám sát của dự án sử dụng MQTT. Lỗi của một phiên Native API cần được kiểm tra riêng, không thể chỉ từ thông báo đó kết luận kết nối MQTT đã mất. Các tính năng quản trị hỗ trợ việc cấu hình, nạp và theo dõi thiết bị; chúng không thay thế broker, API hoặc cơ sở dữ liệu ứng dụng.

Vận hành dịch vụ và kiểm tra khi kết nối gián đoạn

Quy trình khởi động được sắp xếp theo quan hệ phụ thuộc. Đặc biệt, MySQL cần sẵn sàng trước khi backend nhận dữ liệu MQTT vì mã hiện tại chờ ghi cơ sở dữ liệu rồi mới broadcast. Broker có thể chuyển các trạng thái retained ngay sau khi backend đăng ký, nên việc chuẩn bị cơ sở dữ liệu từ đầu giúp hạn chế lỗi ở thời điểm kết nối. Ollama cần sẵn sàng trước khi thử nhánh hỏi đáp AI.

Bảng 3.10 Trình tự khởi động và dấu hiệu sẵn sàng

Bước	Thành phần	Dấu hiệu kiểm tra
1	Mạng LAN và nguồn mô hình	Máy chủ có IP phù hợp; ESP32 có WiFi và được cấp nguồn ổn định.
2	MySQL80	Dịch vụ chạy; mở được nha_thong_minh và đọc được hai bảng dữ liệu.
3	Docker và Mosquitto	Container mosquitto chạy; có ánh xạ 1883; MQTT Explorer kết nối được.
4	Ollama	/api/tags có llama3.1:latest; nhánh /api/chat có thể phản hồi.
5	Backend Node.js	REST cổng 3000 và WebSocket 8081 lắng nghe; log cho thấy đã kết nối broker.
6	Frontend Vite	Mở localhost:5173; tải được lịch sử và có kết nối WebSocket.
7	Đối chiếu toàn tuyến	Quan sát một /state trong MQTT Explorer, hàng dữ liệu tương ứng và thẻ trên dashboard.

Mỗi dấu hiệu trả lời một câu hỏi khác nhau: cổng lắng nghe cho biết có tiến trình nhận kết nối; MQTT Explorer cho biết broker nhận và phân phối bản tin; truy vấn MySQL cho biết dữ liệu đã được lưu; dashboard cho biết ứng dụng đã diễn giải và trình bày được giá trị. Kiểm tra đủ chuỗi sẽ có ý nghĩa hơn việc chỉ quan sát nhãn kết nối ở một màn hình.

Bảng 3.11 Khoanh vùng lỗi mạng và dịch vụ

Hiện tượng	Phạm vi cần kiểm tra	Cách đối chiếu
Không thấy topic từ ESP32	WiFi, IP broker, cổng 1883, tường lửa	Kiểm tra ESP32 cùng LAN; so IP trong YAML với IP máy chủ; dùng MQTT Explorer theo dõi đúng nhánh.
Broker có dữ liệu, web không đổi	Backend, MySQL hoặc WebSocket	Xem log MQTT của backend; tìm hàng mới trong cam_bien_data; kiểm tra phiên WS 8081.
Web có số liệu nhưng không có bản tin mới	Dữ liệu REST cũ hoặc retained	Đối chiếu thời điểm từng mẫu với hoạt động thiết bị; nhãn kết nối WS chưa thể hiện độ mới của cảm biến.
Gửi relay nhận 200 nhưng trạng thái chưa đổi	Đường lệnh hoặc thiết bị chấp hành	Quan sát relay/set rồi relay/state; kiểm tra chế độ vận hành, nguồn và GPIO trên mô hình.
API lịch sử lỗi, broker vẫn chạy	MySQL và xử lý truy vấn	Kiểm tra MySQL80, cấu hình DB_HOST/DB_NAME, quyền kết nối và tên bảng.
Chỉ hỏi đáp AI bị lỗi	Ollama, mô hình hoặc ngữ cảnh MySQL	Kiểm tra 11434, /api/tags, bộ nhớ máy và log backend; phân biệt nhánh regex với nhánh LLM.
Máy chủ mở được, điện thoại không mở được	Phạm vi lắng nghe và địa chỉ frontend	Kiểm tra host của Vite, IP đích API/WS, cùng LAN và tường lửa; localhost trên điện thoại chỉ điện thoại.

Khi WebSocket đóng, frontend tự tạo lại kết nối sau 3 giây. MQTT.js có cơ chế kết nối lại và backend đăng ký nhánh phòng khách trong sự kiện connect. Tuy nhiên, phiên bản hiện tại chưa có hàng đợi lưu ngoại tuyến, mã bản tin để khử lặp, hay bước đồng bộ lại toàn bộ lịch sử sau mỗi lần WebSocket nối lại. Do đó, kết nối phục hồi không đồng nghĩa với việc mọi sự kiện trong thời gian mất mạng đều đã được bổ sung.

Từ thiết kế hiện tại, hướng hoàn thiện phù hợp là thêm thời điểm đo và mã bản tin ở thiết bị, chỉ mục topic–thời gian cho dữ liệu lịch sử, kiểm tra sức khỏe từng dịch vụ, xử lý lỗi ghi cơ sở dữ liệu và trạng thái thiết bị trực tuyến. Đối với mạng, cần giới hạn các cổng được phép truy cập, bổ sung xác thực broker và API, rồi mới mở rộng đến truy cập ngoài LAN. Những thay đổi này giữ nguyên luồng chức năng của đề tài nhưng giúp vận hành và chẩn đoán rõ ràng hơn.

Giao diện web và tổ chức các chức năng

Giao diện web là nơi người dùng quan sát và điều khiển toàn bộ mô hình phòng khách. Những giá trị thu được từ cảm biến, trạng thái relay, lịch sử môi trường và thông tin thói quen được đưa về cùng một ứng dụng, giúp người sử dụng theo dõi ngôi

nhà mà không phải mở riêng từng công cụ kỹ thuật. Nhóm xây dựng giao diện theo hướng dễ nhận biết trạng thái, ít thao tác và có thể mở rộng khi bổ sung thiết bị.

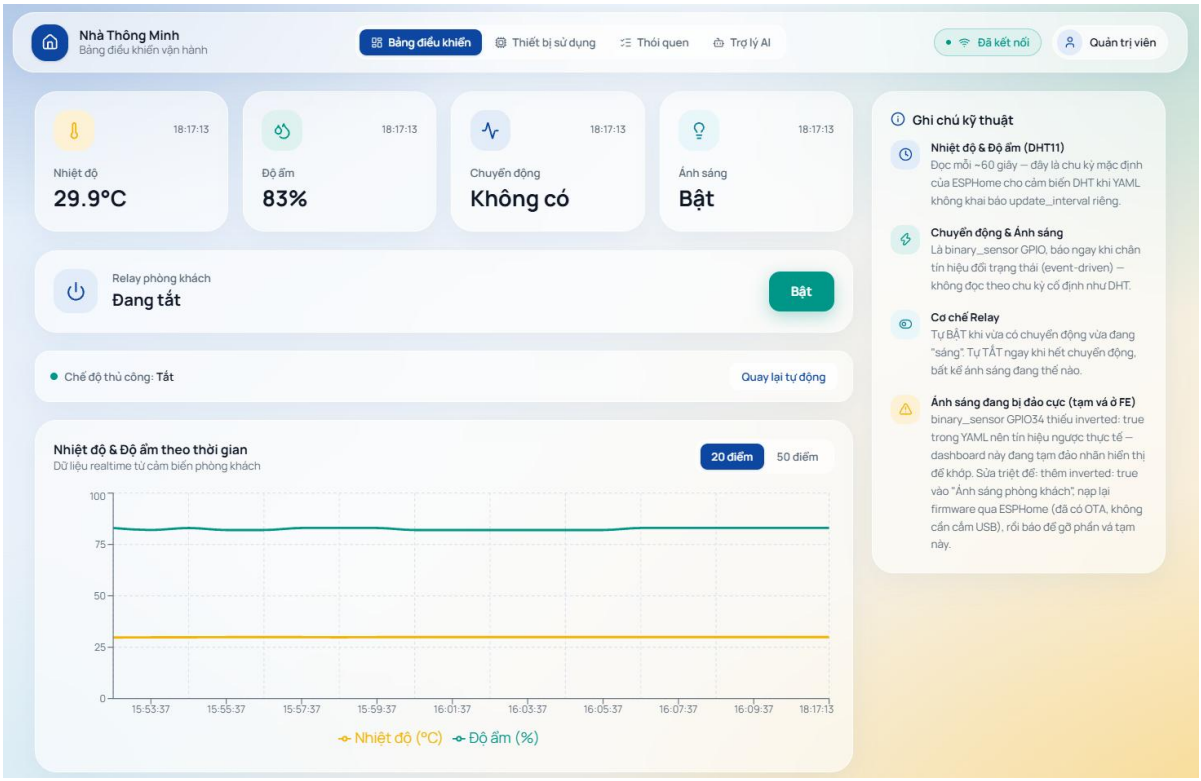
Thanh điều hướng gồm bốn mục: Bảng điều khiển, Thiết bị sử dụng, Thói quen và Trợ lý AI. Cách phân chia này tương ứng với bốn nhu cầu chính: theo dõi tình trạng phòng, tìm hiểu phần cứng, lưu thông tin sinh hoạt và hỏi đáp. Các trang dùng chung ngôn ngữ thiết kế với nền sáng, thẻ nội dung bo góc, biểu tượng trực quan và màu nhấn nhất quán.

Các ảnh minh họa dưới đây được chụp từ ứng dụng của nhóm. Dữ liệu lịch sử được sử dụng lại để thể hiện đầy đủ các thành phần của màn hình trong quá trình trình diễn. Phần mô tả đi từ mục đích của mỗi trang đến thao tác của người dùng và kết quả hệ thống trả về.

Bảng điều khiển giám sát tổng hợp

Bảng điều khiển là màn hình đầu tiên khi truy cập website. Bốn thẻ ở phía trên trình bày nhiệt độ, độ ẩm, chuyển động và ánh sáng; mỗi thẻ có biểu tượng, giá trị hoặc nhãn trạng thái và thời gian cập nhật. Người sử dụng có thể nhận biết nhanh đặc điểm môi trường trong phòng trước khi xem sâu hơn phần lịch sử.

Ngay bên dưới là khu vực điều khiển relay và trạng thái chế độ thủ công. Biểu đồ và ô hỏi trợ lý được đặt tiếp theo để người dùng vừa theo dõi biến động, vừa đặt câu hỏi khi cần giải thích dữ liệu. Nhãn kết nối trên thanh điều hướng phản ánh kết nối giữa trình duyệt với backend; thời điểm cập nhật trên từng thẻ giúp bổ sung ngữ cảnh khi đang xem lại dữ liệu cũ.



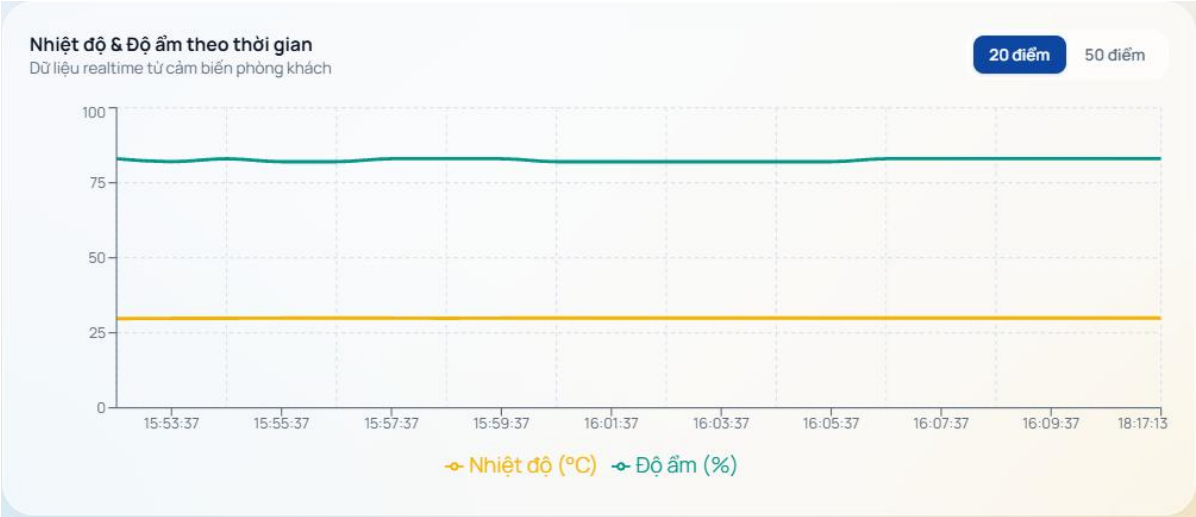
Hình 3.9 Màn hình Bảng điều khiển của website nhà thông minh

Khi tải trang, frontend lấy dữ liệu gần nhất qua REST API rồi tiếp tục nhận các bản tin WebSocket. Nhờ cơ chế này, các thẻ có thể hiển thị giá trị đã lưu ngay khi người dùng mở website, sau đó được thay đổi khi có thông tin mới. Trong quá trình trình diễn, dữ liệu cũ vẫn giúp làm rõ cách bố trí và mối liên hệ giữa các thành phần của hệ thống.

Biểu đồ theo dõi nhiệt độ và độ ẩm

Biểu đồ chuyển các giá trị rời rạc thành một diễn biến theo thời gian, giúp người dùng quan sát xu hướng thay đổi của môi trường. Đường màu vàng biểu diễn nhiệt độ, còn đường màu xanh biểu diễn độ ẩm. Chú giải ghi rõ đơn vị để phân biệt hai đại lượng khi cùng xuất hiện trong một khung đồ thị.

Hai lựa chọn 20 điểm và 50 điểm cho phép thay đổi số mốc gần nhất được hiển thị. Với 20 điểm, giao diện tập trung vào đoạn dữ liệu ngắn để dễ quan sát; với 50 điểm, người dùng có thêm bối cảnh lịch sử. Chức năng này sử dụng số điểm dữ liệu, không tương đương một khoảng thời gian cố định nếu các mẫu được ghi không đều.



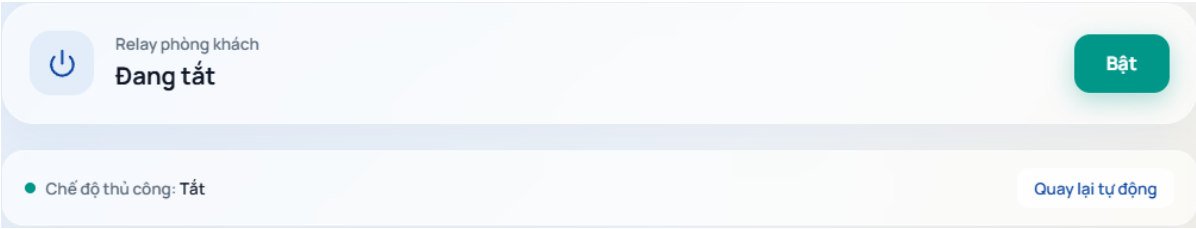
Hình 3.10 Biểu đồ nhiệt độ và độ ẩm với dữ liệu lịch sử

Khi phân tích một thời điểm trên đồ thị, người dùng đối chiếu trục thời gian, màu của đường và đơn vị trong chú giải. Dữ liệu được ghép từ hai topic độc lập và giới hạn trong 50 mốc gần nhất ở frontend. Đối với các phiên bị gián đoạn, cần xem thời gian của từng mẫu để diễn giải đúng sự thay đổi giữa các điểm.

Điều khiển relay và lựa chọn chế độ hoạt động

Khối relay thể hiện trạng thái đóng cắt và cung cấp một nút chuyển trạng thái. Khi relay đang tắt, nút có nhãn Bật; khi đang bật, nút đổi thành Tắt. Sau thao tác, giao diện chuyển sang trạng thái chờ và cập nhật khi nhận được bản tin relay/state qua WebSocket. Cách phản hồi này giúp người dùng theo dõi tiến trình xử lý ngay tại nơi thao tác.

Chế độ thủ công được tách thành một khu vực riêng để cho biết quyền điều khiển đang được ưu tiên. Khi điều khiển LED hoặc còi bằng câu lệnh, backend gửi yêu cầu bật chế độ thủ công trước khi gửi lệnh đầu ra. Nút Quay lại tự động giúp người sử dụng trả quyền xử lý về các quy tắc cảm biến sau khi kết thúc thao tác thủ công.



Hình 3.11 Khu vực điều khiển relay và chế độ vận hành

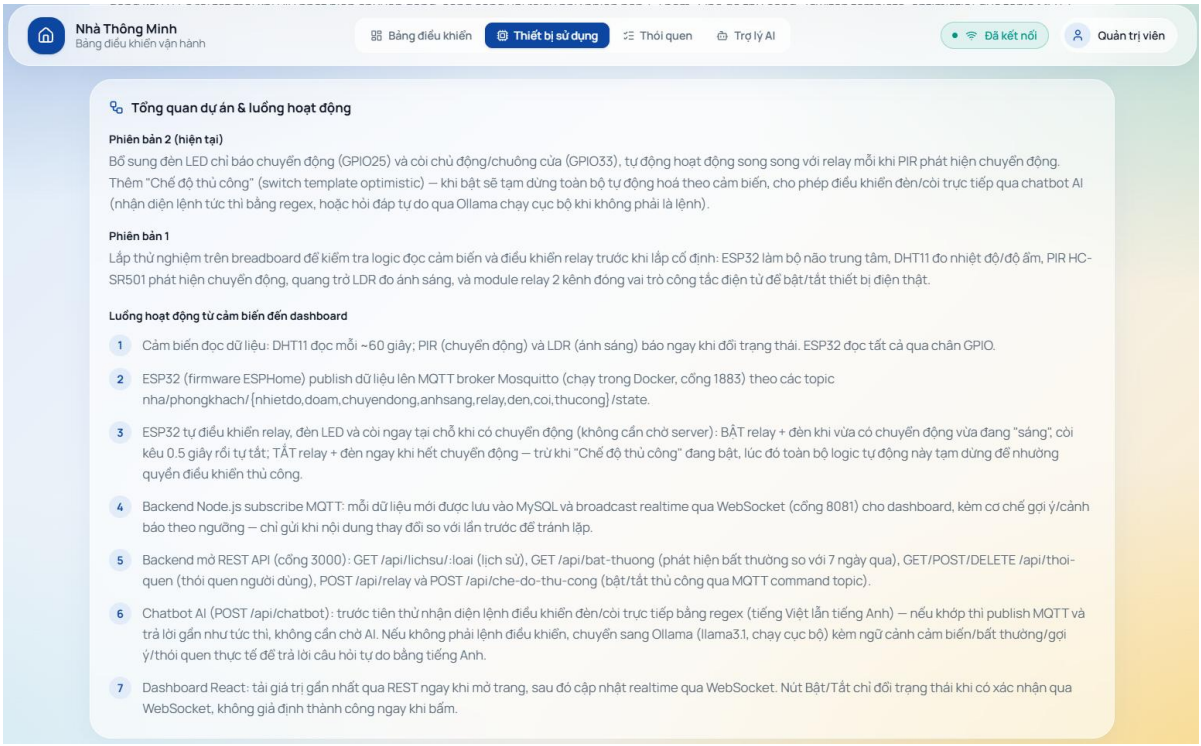
Trạng thái relay trên web tương ứng với công tắc ở GPIO26; lệnh đèn trong chatbot điều khiển LED GPIO25. Việc thể hiện rõ vai trò của hai đầu ra giúp tránh

nhầm lẫn khi trình diễn. Trong các lần thử nghiệm của nhóm, thao tác bật tắt được đối chiếu với trạng thái MQTT và quan sát trên mô hình.

Trang giới thiệu thiết bị sử dụng

Trang Thiết bị sử dụng trình bày các linh kiện theo nhóm chức năng, gồm bộ xử lý trung tâm, cảm biến, cơ cấu chấp hành và linh kiện hỗ trợ. Mỗi thẻ có ảnh thật, tên thiết bị và mô tả vai trò trong hệ thống. Đây là cầu nối giữa sản phẩm phần mềm và mô hình phần cứng, giúp người xem hiểu giá trị trên dashboard được tạo ra từ bộ phận nào.

Nhóm cảm biến giới thiệu DHT11, PIR HC-SR501 và module LDR. Phần cơ cấu chấp hành trình bày relay, LED và còi. Thông tin trên từng thẻ gắn với cách sử dụng thực tế của dự án, chẳng hạn DHT11 cung cấp hai đại lượng môi trường, PIR tạo sự kiện chuyển động và relay đóng cắt một kênh tải.



Hình 3.12 Trang Thiết bị sử dụng: tổng quan dự án và luồng hoạt động

Theo dõi các phiên bản lắp ráp

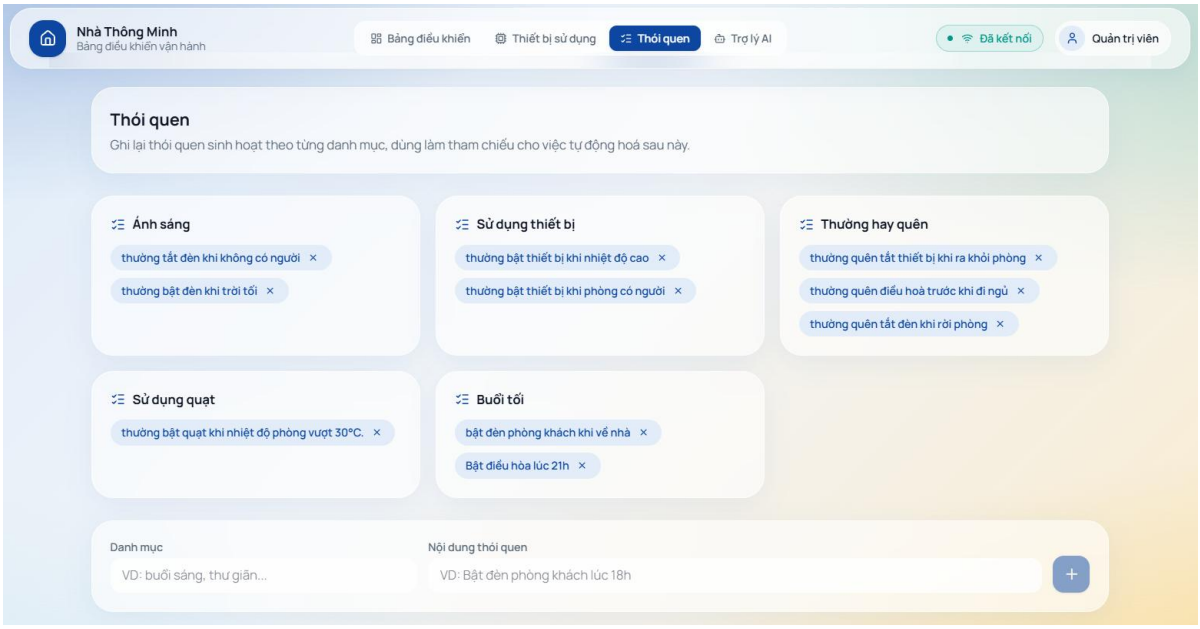
Bộ lọc phiên bản cho phép người dùng xem phần cứng theo từng giai đoạn phát triển. Phiên bản đầu tập trung vào thu thập cảm biến và relay; phiên bản sau bổ sung LED, còi và chế độ thủ công. Chức năng lọc đi cùng ảnh lắp ráp và ghi chú thay đổi, giúp người đọc theo dõi quá trình hoàn thiện mô hình thay vì chỉ quan sát kết quả cuối cùng.

Trong quá trình thuyết trình, trang này có thể được dùng để chỉ ra vị trí các linh kiện, lý do bổ sung đầu ra và mối liên hệ với các chân GPIO đã thiết kế. Việc lưu ảnh theo phiên bản cũng giúp thuận tiện đối chiếu khi kiểm tra dây nối hoặc tái lắp mô hình.

Trang quản lý thói quen sinh hoạt

Trang Thói quen cho phép lưu các nội dung sinh hoạt theo danh mục. Những mục đã có được hiển thị thành từng nhóm, mỗi nội dung là một nhãn ngắn kèm nút xóa. Khu vực nhập phía dưới gồm tên danh mục, nội dung thói quen và nút thêm, tạo một quy trình thao tác rõ ràng ngay trên cùng màn hình.

Người dùng có thể chọn một danh mục đã tồn tại hoặc nhập tên mới, sau đó ghi nội dung cần lưu. Khi thêm thành công, mục vừa tạo được đưa vào nhóm tương ứng; khi xóa một mục, danh sách trên trang được cập nhật. Dữ liệu nằm trong bảng thoi_quen_nguoi_dung nên vẫn có thể sử dụng sau khi đóng và mở lại trình duyệt.



Hình 3.13 Màn hình quản lý thói quen theo danh mục

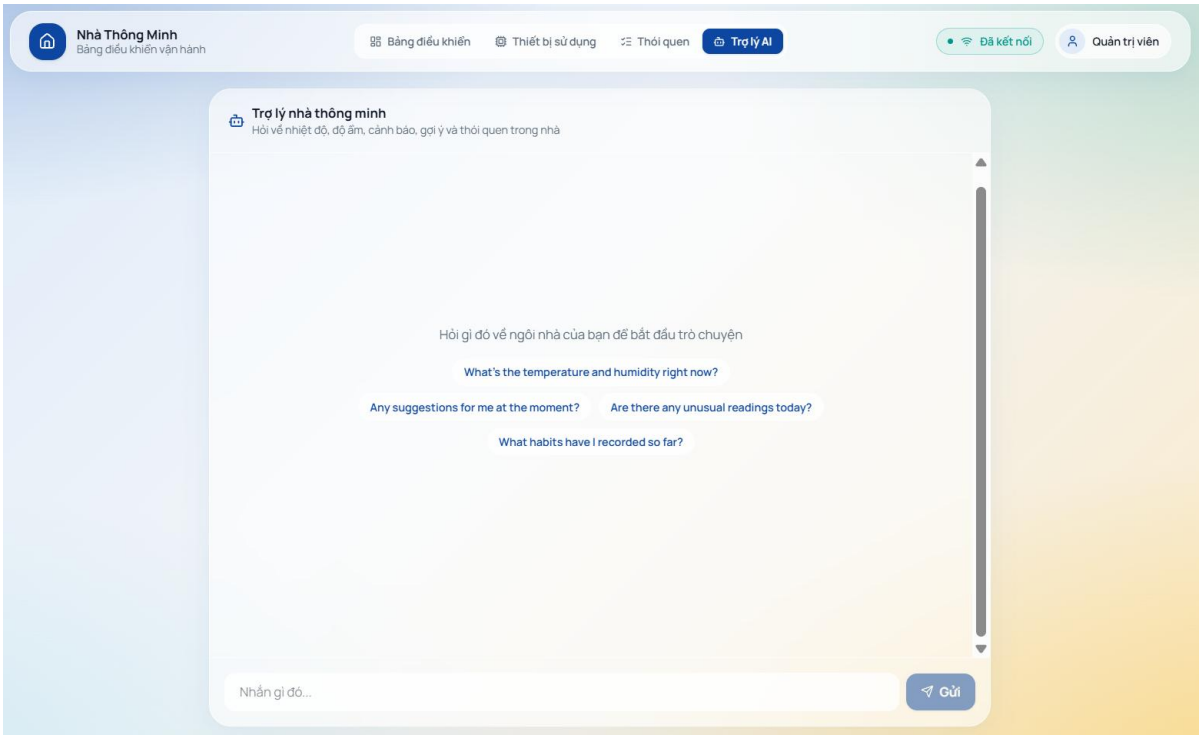
Trong dữ liệu của dự án có 10 mục thuộc 5 danh mục. Các nội dung này được backend đưa vào ngữ cảnh để trợ lý có thể trả lời các câu hỏi liên quan đến thói quen đã ghi. Chức năng hiện tập trung vào ghi nhớ và tham chiếu thông tin; việc tự chuyển một nội dung thành lịch bật tắt thiết bị là hướng mở rộng tiếp theo.

Trang hội thoại với trợ lý AI

Trang Trợ lý AI cung cấp một không gian hội thoại riêng, gồm các tin nhắn người dùng, câu trả lời của trợ lý và vùng nhập câu hỏi. Khi chưa có hội thoại, các câu hỏi

gợi ý giúp người dùng bắt đầu với những nội dung hệ thống đang hỗ trợ, như nhiệt độ và độ ẩm gần nhất, gợi ý môi trường, bất thường và thói quen đã lưu.

Người dùng nhập câu hỏi rồi chọn Gửi. Trong lúc backend xử lý, giao diện thể hiện trạng thái đang suy nghĩ; khi có kết quả, câu trả lời xuất hiện thành một tin nhắn riêng. Cách trình bày theo lượt hội thoại giúp người dùng dễ đối chiếu câu hỏi với phản hồi và thuận tiện hơn khi trình diễn nhiều tình huống liên tiếp.



Hình 3.14 Giao diện Trợ lý AI với ô nhập và các câu hỏi gợi ý

Với câu hỏi về nhiệt độ và độ ẩm, backend lấy giá trị gần nhất từ MySQL rồi gửi kèm vào ngữ cảnh cho Ollama. Với các lệnh LED hoặc còi được nhận diện, backend thực hiện nhánh quy tắc và trả lời xác nhận đã gửi lệnh. Như vậy, cùng một giao diện hội thoại hỗ trợ cả khai thác thông tin và điều khiển một số đầu ra, trong khi hai luồng xử lý bên trong vẫn được phân định rõ.

Lịch sử tin nhắn trên trang phục vụ việc theo dõi phiên trao đổi ở frontend. Mỗi lần gọi Ollama hiện gửi ngữ cảnh hệ thống và câu hỏi mới; phiên bản này chưa gửi lại đầy đủ các lượt hội thoại trước đó cho mô hình. Khi dữ liệu lịch sử được sử dụng để minh họa, giá trị trong câu trả lời được đối chiếu với tập dữ liệu đã lưu.

Hiển thị gợi ý và cảnh báo trên giao diện

Ngoài các giá trị cảm biến, dashboard còn có vùng thông báo để giải thích những trạng thái cần người dùng chú ý. Khi nhiệt độ hoặc độ ẩm đi qua các ngưỡng đã khai

báo, backend tạo nội dung gợi ý, xác định mức thông báo và gửi đến frontend. Màu sắc của thông báo giúp người sử dụng nhận biết mức độ, còn câu mô tả cho biết giá trị và hành động tham khảo.

Các thông báo được bổ sung vào đầu danh sách và có nút đóng để người dùng chủ động thu gọn màn hình. Frontend giữ tối đa 5 thông báo gần nhất. Nội dung gợi ý không tự động bật điều hòa, quạt hoặc máy tạo ẩm; các thiết bị đó được nhắc trong câu gợi ý và chưa phải các đầu ra đã cấu hình trong mô hình.

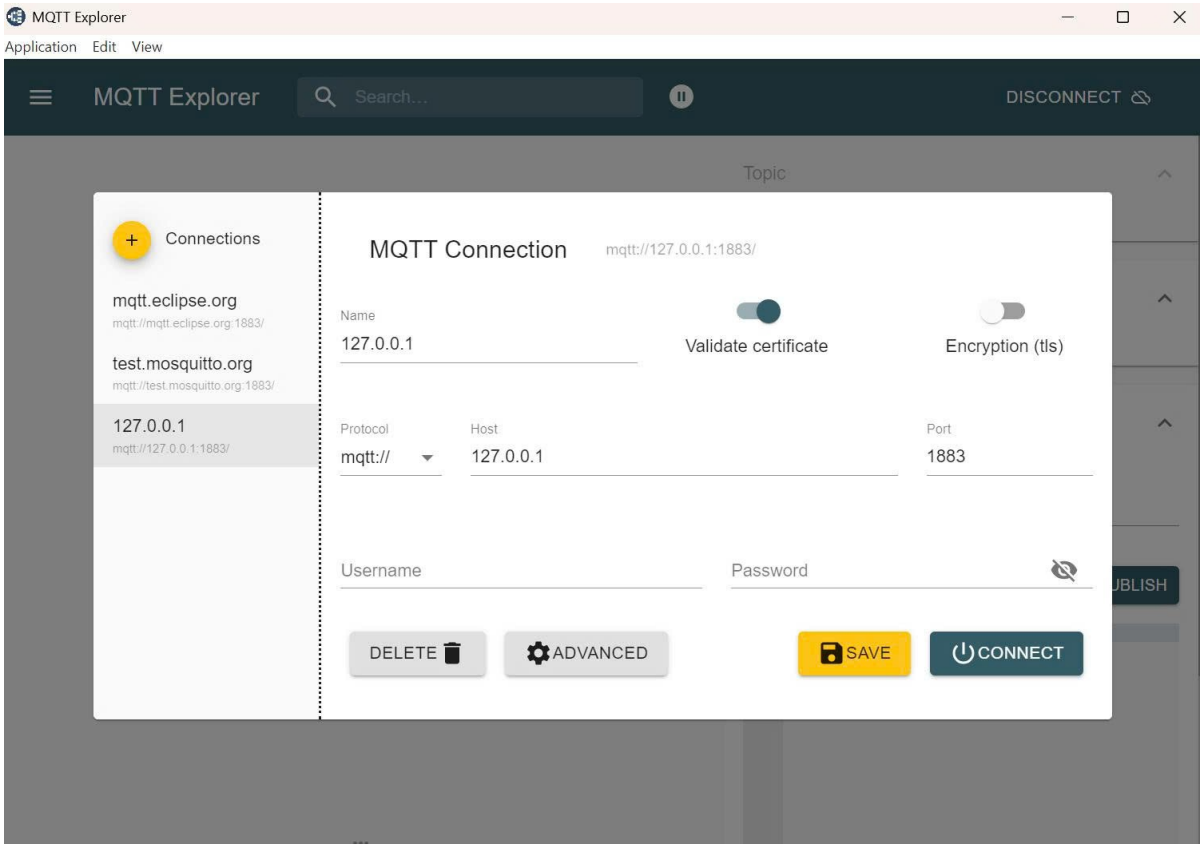
Công cụ cấu hình và kiểm chứng kết nối

Bên cạnh website dành cho người dùng, nhóm sử dụng MQTT Explorer và ESPHome Device Builder để thiết lập, quan sát và đối chiếu hoạt động bên trong hệ thống. Website cho thấy kết quả ở tầng ứng dụng; MQTT Explorer cho thấy topic và payload trên broker; ESPHome Device Builder thể hiện cấu hình của nút ESP32. Ba góc nhìn bổ sung cho nhau trong quá trình lắp ráp và thử nghiệm.

Thiết lập kết nối bằng MQTT Explorer

Ảnh cấu hình của nhóm cho thấy MQTT Explorer kết nối tới broker trên máy cục bộ với giao thức mqtt://, địa chỉ 127.0.0.1 và cổng 1883. Sau khi kết nối, công cụ hiển thị các nhánh topic mà broker chuyển đến. Đây là bước kiểm tra trực tiếp đường truyền trước khi đối chiếu dữ liệu trên dashboard.

Trong quy trình thử nghiệm, nhóm kiểm tra lần lượt địa chỉ broker, cổng dịch vụ và topic đang được công bố. Khi dashboard chưa thay đổi như mong muốn, việc quan sát MQTT Explorer giúp xác định bản tin có xuất hiện trên broker hay không, qua đó thu hẹp phạm vi kiểm tra giữa thiết bị, máy chủ và giao diện.



Hình 3.15 Cấu hình kết nối broker cục bộ trên MQTT Explorer

Quan sát cây topic và các giá trị cảm biến

Nhánh nha/phongkhach tổ chức dữ liệu theo từng loại: nhietdo, doam, chuyendong, anhsang và relay. Trong ảnh do nhóm ghi lại, nhiệt độ hiển thị 32,5°C, độ ẩm 59%, chuyển động OFF, ánh sáng ON và relay OFF. Mỗi giá trị nằm trong một topic trạng thái riêng, phù hợp với giao ước mà backend sử dụng.

Cây topic giúp kiểm tra nhanh cả cấu trúc đặt tên lẫn nội dung payload. Khi nhiệt độ hoặc độ ẩm thay đổi ở thiết bị, người kiểm thử có thể quan sát bản tin tại đây rồi đối chiếu giá trị tương ứng trên website. Ảnh thể hiện một thời điểm trong quá trình triển khai, vì vậy các giá trị minh họa có thể khác với bộ dữ liệu được nạp lại khi chụp các màn hình web.

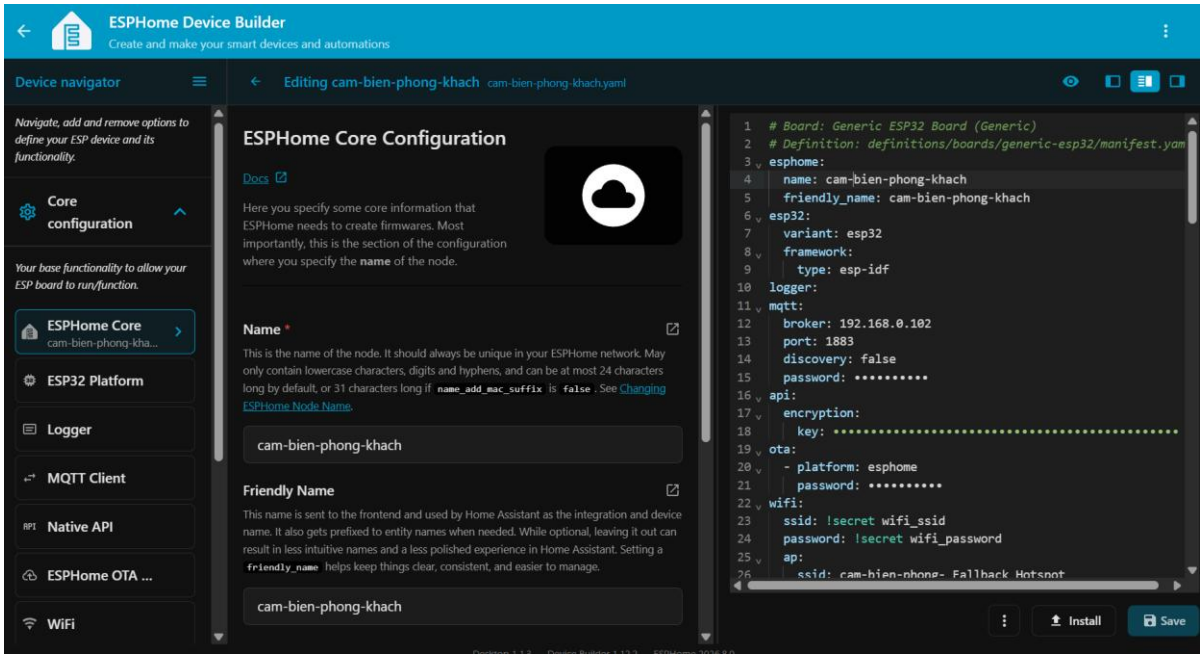


Hình 3.16 Nhánh topic phòng khách và dữ liệu ghi nhận trên MQTT Explorer

Xây dựng cấu hình bằng ESPHome Device Builder

ESPHome Device Builder cho phép quản lý các thành phần của nút cam-bien-phong-khach theo nhóm như ESPHome Core, ESP32 Platform, Logger, MQTT Client, Native API, OTA và WiFi. Vùng cấu hình ở bên phải hiển thị YAML tương ứng, giúp nhóm đối chiếu các tham số của thiết bị trong cùng một không gian làm việc.

Ảnh chụp thể hiện tên nút, biến thể ESP32, framework esp-idf và địa chỉ broker. Các trường mật khẩu và khóa được che trên giao diện. Từ cấu hình này, nhóm bổ sung cảm biến, công tắc và các hành động tự động đã trình bày trong các phần trước. Cấu hình đầy đủ đã tách thông tin xác thực được đặt ở Phụ lục A để thuận tiện tham khảo.



Hình 3.17 Giao diện ESPHome Device Builder dùng để cấu hình nút ESP32

Việc kết hợp ảnh mô hình, các màn hình website và ảnh công cụ kỹ thuật giúp mô tả sản phẩm ở cả ba mức: thiết bị vật lý, truyền thông và tương tác người dùng. Qua đó, quá trình triển khai được thể hiện bằng các thành phần có thể quan sát, thay vì chỉ dừng ở mô tả kiến trúc hoặc danh sách chức năng.

CHƯƠNG 4 KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ

Môi trường và phạm vi đánh giá

Các thử nghiệm trước của nhóm được thực hiện trên mô hình ESP32 với các cảm biến và cơ cấu chấp hành đã trình bày. Nhóm đã chạy thành công các chức năng đọc cảm biến, truyền MQTT, giám sát trên web, điều khiển tự động và thủ công, lưu lịch sử, quản lý thói quen và hỏi đáp AI. Dữ liệu còn lưu trong MySQL được sử dụng để thống kê và minh họa kết quả.

Ngày 15/10/2026, các dịch vụ phần mềm được khởi động lại để kiểm tra khả năng truy cập dữ liệu và phản hồi. Broker Mosquitto, backend, dashboard và Ollama được chạy; MySQL sử dụng phiên bản 8.0.44. Phần đánh giá ở lần khởi động lại gồm API đọc lịch sử, đầu vào không hợp lệ, kết nối WebSocket, các hàm quy tắc và chatbot. Việc gửi thêm các lệnh có thể làm thay đổi cơ cấu chấp hành không được dùng làm nguồn thống kê cho dữ liệu lịch sử.

Nhóm sử dụng lại trạng thái cũ để trình bày giao diện khi cần minh họa. Hai phiên dữ liệu chính ngày 01/09/2026 và 08/09/2026 được phân tích độc lập với bản tin retained nhận lại vào tháng 10. Số lượng các hàng mô tả dữ liệu lưu trữ, không được chuyển thành tỷ lệ thành công hoặc độ chính xác cảm biến nếu chưa có quy trình đo tương ứng.

Bảng 4.1 Các nguồn dùng để đánh giá

Nguồn	Nội dung	Cách diễn giải
Thử nghiệm mô hình của nhóm	Các chức năng cảm biến và điều khiển đã chạy thành công	Kết quả chức năng của các lần chạy trước
MySQL lịch sử	2.077 hàng trong hai ngày 01/09 và 08/09/2026	Thống kê bản tin, phạm vi giá trị và ví dụ lệnh phản hồi
Cấu hình và mã nguồn	YAML ESPHome, backend và frontend	Xác định thuật toán, điều kiện, topic và API
Kiểm tra ngày 15/10/2026	MQTT, REST, WS, logic, build, chatbot	Khả năng hoạt động phần mềm trong lần kiểm tra lại

Kết quả thống kê dữ liệu lịch sử

Tập dữ liệu lịch sử bắt đầu lúc 13:35:51 ngày 01/09/2026 và kết thúc lúc 16:10:37 ngày 08/09/2026 theo giờ Việt Nam. Có 849 hàng trong ngày 01/09 và 1.228 hàng

trong ngày 08/09, tổng cộng 2.077 hàng. Dữ liệu nằm trong hai phiên chạy; khoảng cách bảy ngày giữa mốc đầu và mốc cuối không thể được hiểu là bảy ngày giám sát liên tục.

Trong tổng số hàng, 1.950 hàng có topic kết thúc bằng /state, còn 127 hàng có topic kết thúc bằng /set. Bản tin chuyển động có số lượng lớn nhất vì được gửi theo sự kiện, trong khi DHT11 cung cấp hai kênh theo chu kỳ. Các bản tin khởi tạo, retained hoặc công bố lại trạng thái cũng có thể góp phần vào số lượng hàng.

Bảng 4.2 Số bản ghi theo topic trong hai phiên lịch sử

Topic sau nha/phongkhach/	Số hàng	Loại
anhsang/state	145	Trạng thái
chuyendong/state	699	Trạng thái
coi/set	6	Lệnh
coi/state	118	Trạng thái
den/set	14	Lệnh
den/state	127	Trạng thái
doam/state	303	Trạng thái
nhietdo/state	304	Trạng thái
relay/set	77	Lệnh
relay/state	240	Trạng thái
thucong/set	30	Lệnh
thucong/state	14	Trạng thái
Tổng cộng	2.077	1.950 trạng thái và 127 lệnh

Có 10 mục thói quen thuộc 5 danh mục trong cơ sở dữ liệu. Các danh mục gồm Buổi tối, Thường hay quên, Sử dụng quạt, Sử dụng thiết bị và Ánh sáng. Chức năng này cung cấp nội dung tham chiếu cho chatbot; việc có một câu mô tả giờ bật thiết bị trong bảng không đồng nghĩa một lịch tự động đã được tạo.

Đặc điểm số liệu nhiệt độ và độ ẩm

Bảng 4.3 Thống kê mô tả nhiệt độ và độ ẩm

Đại lượng	Số hàng	Nhỏ nhất	Lớn nhất	Trung bình
Nhiệt độ	304	29,4°C	35,0°C	31,251°C
Độ ẩm tương đối	303	54%	88%	69,033%

Trung bình trong bảng được tính trên toàn bộ các hàng tương ứng trong hai phiên lịch sử, với mỗi hàng có cùng trọng số. Đây là trung bình các mẫu đã ghi, không phải trung bình theo thời lượng hoạt động của cả khoảng ngày. Số hàng nhiệt độ và độ ẩm khác nhau một đơn vị nên cũng không thể giả định hai đại lượng luôn có đầy đủ cặp tại mọi thời điểm.

Khoảng nhiệt độ đi qua cả ngưỡng gợi ý trên 30°C và ngưỡng cảnh báo trên 33°C. Độ ẩm lớn nhất vượt ngưỡng 80% được khai báo trong backend. Vì vậy, dữ liệu đã lưu cung cấp ví dụ phù hợp để minh họa các mức gợi ý của hệ thống. Không có thiết bị đo chuẩn đi kèm trong tập dữ liệu để tính sai số tuyệt đối hay sai số phần trăm của DHT11.

Giá trị nhiệt độ và độ ẩm ở cuối phiên ngày 08/09 là 29,9°C và 83%. Khi broker được bật lại vào ngày 15/10, các giá trị này còn trong các bản tin retained và được backend nhận lại. Chúng được dùng để hiển thị giao diện và kiểm tra hỏi đáp theo dữ liệu đã có, thay vì coi đó là một cặp đo mới tại phòng khách ngày 15/10.

Minh họa luồng lệnh và phản hồi từ dữ liệu cũ

Trong lịch sử ngày 08/09 có các chuỗi lệnh LED và relay kèm trạng thái trả về. Ví dụ, lúc 16:02:13 backend ghi nhận thucong/set bằng ON và den/set bằng ON, sau đó den/state bằng ON. Lúc 16:02:24 xuất hiện den/set bằng OFF và den/state bằng OFF. Các hàng này phù hợp với luồng điều khiển thủ công qua MQTT đã thiết kế.

Bảng 4.4 Một số sự kiện lệnh và trạng thái ngày 08 tháng 09

Giờ địa phương	Topic sau nha/phongkhach/	Payload	Ý nghĩa
16:02:13	thucong/set	ON	Yêu cầu chế độ thủ công
16:02:13	den/set	ON	Lệnh bật LED
16:02:13	den/state	ON	Trạng thái LED được công bố

Giờ địa phương	Topic sau nha/phongkhach/	Payload	Ý nghĩa
16:02:24	den/set	OFF	Lệnh tắt LED
16:02:24	den/state	OFF	Trạng thái LED được công bố
16:03:18	relay/set	ON	Lệnh bật relay
16:03:18	relay/state	ON	Trạng thái relay được công bố
16:03:27	relay/set	OFF	Lệnh tắt relay
16:03:27	relay/state	OFF	Trạng thái relay được công bố

Thời gian trong bảng MySQL có độ phân giải đến giây và là thời điểm ghi tại máy chủ. Nhiều lệnh và phản hồi nằm trong cùng một giây nên bảng chỉ chứng minh trật tự bản ghi, không cho phép kết luận độ trễ bằng 0 ms. Ngoài ra, trạng thái GPIO hoặc relay ở mức phần mềm không thay thế phép đo điện trên tải ngoài.

Một số lệnh relay được ghi lặp hai lần trong cùng giây. Điều này cho thấy việc đếm số hàng /set chưa đủ để suy ra số thao tác duy nhất của người dùng. Để đo tỷ lệ xác nhận và độ trễ điều khiển một cách chắc chắn hơn, phiên bản sau cần mã lệnh, thời điểm gửi, thời điểm thiết bị nhận và phản hồi tương ứng.

Kết quả thử nghiệm chức năng của mô hình

Nhóm đã thực hiện thành công các kịch bản chức năng chính trong các lần chạy trước. Bảng dưới tổng hợp theo từng chức năng; phần số liệu và các bản tin minh họa ở trên bổ sung bằng chứng cho lưu trữ, trạng thái và một số thao tác điều khiển. Bảng không biểu diễn một thử nghiệm thống kê với số lần lặp cố định.

Bảng 4.5 Tổng hợp các kịch bản chức năng đã thử nghiệm

Kịch bản	Kết quả thực hiện	Nhận xét
Đọc DHT11	Thành công	Có dữ liệu nhiệt độ và độ ẩm trong MySQL
Đọc PIR và LDR	Thành công	Có bản tin ON/OFF theo từng topic
PIR kích LED và còi	Thành công	Đã chạy mô hình; cấu hình còi có khoảng chờ 500 ms
Relay tự động	Thành công	Thực hiện theo trạng thái PIR, ánh sáng và chế độ

Kịch bản	Kết quả thực hiện	Nhận xét
Bật tắt relay từ dashboard	Thành công	Có lệnh và trạng thái relay trong lịch sử
Điều khiển LED, còi bằng câu lệnh	Thành công	Có den/set, coi/set và các trạng thái tương ứng
Chuyển chế độ thủ công	Thành công	Có thucong/set và thucong/state
Lưu và đọc lịch sử	Thành công	Các phiên dữ liệu được giữ trong MySQL
Quản lý thói quen	Thành công	Có 10 mục thuộc 5 danh mục
Hỏi đáp AI	Thành công	Trả lời theo dữ liệu cảm biến và ngữ cảnh

Kiểm tra lại phần mềm ngày 15 tháng 10 năm 2026

Năm API lịch sử đều trả HTTP 200 và 50 hàng mới nhất cho từng loại hỗ trợ. Loại không có trong ánh xạ trả HTTP 404. Hai API điều khiển relay và chế độ thủ công trả HTTP 400 khi nhận trạng thái INVALID, phù hợp với ràng buộc ON/OFF. Kiểm tra đầu vào không hợp lệ không tạo thêm lệnh điều khiển.

Kết nối WebSocket tới cổng 8081 được mở thành công. Một MQTT client quan sát nhận 8 topic trạng thái với cờ retain bằng true và QoS bằng 0. Backend đồng thời nhận lại các trạng thái được broker giữ, làm tăng cơ sở dữ liệu từ 2.077 lên 2.085 hàng trong lần khởi động. Tám hàng này được loại khỏi thống kê hai phiên lịch sử.

Quá trình npm run build hoàn thành thành công, tạo gói giao diện trong dist. Công cụ build thông báo một gói JavaScript sau tối ưu lớn hơn ngưỡng khuyến nghị 500 kB; đây là hướng để tối ưu tải trang bằng chia nhỏ mã khi mở rộng ứng dụng, không làm thất bại quá trình build hiện tại.

Bảng 4.6 Kết quả kiểm tra dịch vụ phần mềm

Hạng mục	Kết quả quan sát	Đánh giá
MQTT cổng 1883	Kết nối thành công; nhận 8 trạng thái retained	Đạt kết nối và nhận trạng thái đã lưu
5 API lịch sử	HTTP 200; mỗi loại trả 50 hàng	Đạt
Loại lịch sử không tồn tại	HTTP 404	Đạt kiểm tra đầu vào
Trạng thái điều khiển không hợp lệ	HTTP 400 ở hai API	Đạt kiểm tra đầu vào

Hạng mục	Kết quả quan sát	Đánh giá
API thói quen	HTTP 200; 10 mục, 5 danh mục	Đạt đọc dữ liệu
API bất thường	HTTP 200; danh sách rỗng ở lần gọi	Đạt phản hồi; cần đọc theo cửa sổ dữ liệu
WebSocket cổng 8081	Mở kết nối thành công	Đạt kết nối trình duyệt và backend
Build dashboard	Hoàn thành; có khuyến nghị kích thước gói	Đạt
Chatbot hỏi dữ liệu	HTTP 200; trả đúng 29,9°C và 83%	Đạt trong lần kiểm tra lại

Kiểm tra các ngưỡng và nhận diện câu lệnh

Nhóm đối chiếu trực tiếp hai hàm kiemTraGoiY và phatHienLenh trong backend. Các phép kiểm tra chạy tách biệt phần kết nối phần cứng, do đó không phát lệnh MQTT và không làm thay đổi đầu ra. Tổng cộng 15 trường hợp có kết quả đúng theo điều kiện đã viết trong mã nguồn.

Bảng 4.7 Các trường hợp kiểm tra logic

Đầu vào	Kết quả mong đợi và quan sát
Nhiệt độ 30°C	Không tạo gợi ý
Nhiệt độ 30,1°C	goi_y
Nhiệt độ 33°C	goi_y
Nhiệt độ 33,1°C	canh_bao
Độ ẩm 39,9%	goi_y
Độ ẩm 40%	Không tạo gợi ý
Độ ẩm 80%	Không tạo gợi ý
Độ ẩm 80,1%	goi_y
bật đèn	den: ON
tắt đèn	den: OFF
bật còi	coi: ON

Đầu vào	Kết quả mong đợi và quan sát
tắt còi	coi: OFF
turn on the light	den: ON
turn off the buzzer	coi: OFF
Nhiệt độ hiện tại là bao nhiêu?	Không nhận là lệnh điều khiển

Các giá trị đúng bằng 30°C, 33°C, 40% và 80% giúp xác định rõ dấu lớn hơn hoặc nhỏ hơn trong điều kiện. Kết quả 15/15 chỉ phản ánh các trường hợp cụ thể đã kiểm tra. Nó không chứng minh mọi câu tiếng Việt và tiếng Anh đều được hiểu đúng, cũng không thay cho phép kiểm tra toàn bộ đường điều khiển đến tải vật lý.

Kết quả hỏi đáp AI

Chatbot đã được nhóm sử dụng thành công để hỏi về dữ liệu cảm biến và nội dung tham chiếu của hệ thống. Trong lần kiểm tra lại, câu hỏi “What are the temperature and humidity readings? Answer in one sentence.” nhận phản hồi HTTP 200 với nội dung “The current temperature is 29.9°C, and the humidity is at 83%.” Hai giá trị trùng với trạng thái đã nạp vào backend, thể hiện được luồng lấy dữ liệu, tạo ngữ cảnh và sinh câu trả lời. Màn hình hội thoại ở Chương 3 minh họa cách người dùng tiếp cận chức năng này trên website.

Một lần gọi thành công trong phiên kiểm tra có thời gian chờ khoảng 19,2 giây ở phía chương trình gọi API. Đây là thời gian của một lần hỏi đáp cụ thể, không phải độ trễ trung bình, độ trễ điều khiển hoặc cam kết hiệu năng của hệ thống. Dịch vụ cần thời gian nạp mô hình khi khởi động; một yêu cầu ban đầu chưa thành công và lần kiểm tra lại sau khi mô hình sẵn sàng đã trả lời đúng.

Kết quả cho thấy nhánh lấy ngữ cảnh từ MySQL và gọi Ollama hoạt động. Từ “current” trong câu trả lời hiện do hướng dẫn ngữ cảnh của backend tạo ra; trong trường hợp này, dữ liệu là trạng thái cũ được dùng để minh họa. Hướng hoàn thiện là đưa thời gian đo và độ mới của dữ liệu vào ngữ cảnh để trợ lý trả lời “giá trị ghi nhận gần nhất” khi không có phép đo mới.

Ưu điểm của hệ thống

Hệ thống tạo được một chuỗi IoT hoàn chỉnh từ cảm biến vật lý tới giao diện web và dữ liệu lưu bền. Mỗi khối có vai trò rõ ràng, giúp nhóm quan sát và kiểm tra sự cố

theo đường truyền. Cây topic MQTT và các API nhỏ cho phép bổ sung thành phần mà không phải thay đổi đồng thời toàn bộ kiến trúc.

Logic phản ứng với PIR được đặt tại ESP32 nên có thể thực hiện ở thiết bị khi không chờ backend quyết định. Dữ liệu lịch sử được giữ trong MySQL và có thể sử dụng lại khi phần mềm khởi động. Dashboard kết hợp truy vấn ban đầu và cập nhật WebSocket, hỗ trợ cả theo dõi và tương tác trong cùng một giao diện.

Trợ lý AI bổ sung hình thức khai thác dữ liệu bằng ngôn ngữ tự nhiên. Các lệnh đơn giản đi qua nhánh xác định trước, trong khi phần hỏi đáp dùng một mô hình cục bộ có ngữ cảnh. Cách kết hợp này phù hợp với phạm vi học phần và giúp nhóm nhận diện rõ hơn sự khác nhau giữa điều khiển theo quy tắc, thống kê đơn giản và hỏi đáp bằng LLM.

Giới hạn và hướng hoàn thiện

Về đồng bộ trạng thái, payload hiện chưa có timestamp từ ESP32, mã thiết bị, số thứ tự hoặc cờ dữ liệu cũ trong bản ghi MySQL. Backend cũng chưa lưu retain của MQTT. Vì vậy, dữ liệu phát lại có thể được ghi với thời gian nhận mới. Cần bổ sung các trường nguồn gốc này và hiển thị tuổi dữ liệu để người dùng phân biệt trạng thái tham khảo với trạng thái vừa đo.

Về điều khiển, chế độ tự động được kích theo cạnh PIR, chưa tính lại đầu ra khi chỉ ánh sáng đổi hoặc khi vừa thoát thủ công. Relay được xét sau khoảng chờ 500 ms của còi. Giao diện cần đồng bộ giá trị thủ công khi khởi tạo và thống nhất ánh xạ cực tính của LDR. Các thay đổi này sẽ làm hành vi hệ thống dễ dự đoán hơn trong nhiều tình huống.

Về dữ liệu và thống kê, điều kiện đủ 3 ngày của phần bất thường chưa kiểm tra số ngày có mẫu thực sự; cửa sổ trung bình có thể chỉ chứa dữ liệu phát lại. Chưa có chỉ số đo độ đầy đủ dữ liệu hoặc cơ chế hiệu chuẩn. Khi đánh giá phiên bản tiếp theo, cần có các phiên đo liên tục, nhật ký điều kiện thử và thiết bị tham chiếu nếu muốn kết luận về độ chính xác.

Về độ ổn định và truy cập, backend cần xử lý lỗi cho từng luồng, kiểm tra dữ liệu đầu vào đầy đủ hơn và giới hạn phạm vi CORS. Các giao thức đang dùng mqtt://, http:// và ws:// trong LAN; xác thực API và mã hóa đường truyền cần được bổ sung trước khi cho phép truy cập từ môi trường rộng hơn. Broker và máy chủ hiện cũng là các điểm phụ thuộc chung của hệ thống.

Về trợ lý, cần bổ sung timestamp, lịch sử hội thoại có giới hạn và trạng thái dịch vụ mô hình. Regex nên xử lý phủ định, nhiều hành động và các câu không chắc chắn. Nội dung thói quen chỉ nên trở thành lịch tự động khi có bộ lập lịch và quy tắc xác nhận riêng; mô hình hiện tại chưa thực hiện bước này.

Bảng 4.8 Các hướng phát triển ưu tiên

Ưu tiên	Nội dung	Kết quả mong muốn
1	Chuẩn hóa payload, thời gian đo và retain	Hiển thị đúng độ mới, phân biệt dữ liệu phát lại
2	Đồng bộ chế độ và cực tính ánh sáng	Giảm khác biệt giữa firmware, MQTT và dashboard
3	Bổ sung xác nhận lệnh và timeout	Phân biệt lệnh đã gửi, đã nhận, đã thực thi
4	Xử lý lỗi và chỉ mục dữ liệu	Vận hành ổn định hơn khi dữ liệu tăng
5	Xác thực API và mã hóa phù hợp	Kiểm soát quyền truy cập khi mở rộng
6	Ngữ cảnh AI có thời gian, hội thoại	Trả lời rõ nguồn dữ liệu và bám ngữ cảnh
7	Nhiều phòng và bộ lập lịch	Mở rộng ứng dụng sau khi các khối cơ bản ổn định

KẾT LUẬN

Đề tài đã xây dựng mô hình giám sát và điều khiển nhà thông minh cho phòng khách, kết hợp ESP32, DHT11, PIR, module ánh sáng, relay, LED và còi với các dịch vụ MQTT, Node.js, MySQL và dashboard React. Hệ thống thể hiện được hai luồng chính của một ứng dụng IoT: thu thập trạng thái từ thiết bị lên ứng dụng và gửi lệnh từ ứng dụng về cơ cấu chấp hành.

Các chức năng tự động và thủ công đã được nhóm thử nghiệm thành công. Dữ liệu lịch sử gồm 2.077 hàng cung cấp cơ sở để mô tả phạm vi hoạt động, giá trị môi trường và ví dụ về lệnh, trạng thái phản hồi. Kiểm tra lại các dịch vụ xác nhận khả năng truy vấn lịch sử, kết nối MQTT, WebSocket, build giao diện và hỏi đáp AI; các phép kiểm tra logic tại ngưỡng cho kết quả đúng với mã nguồn.

Phần trợ lý AI sử dụng dữ liệu của hệ thống làm ngữ cảnh để trả lời câu hỏi, đồng thời tách một số lệnh đèn, còi sang nhánh quy tắc. Đây là bước mở rộng giao diện tương tác, chưa phải mô hình tự học hành vi và tự quyết định toàn bộ hoạt động trong nhà. Việc lưu thói quen hiện phục vụ tham chiếu cho hỏi đáp.

Qua quá trình thực hiện, nhóm vận dụng được kiến thức phần cứng, lập trình cấu hình thiết bị, mạng máy tính, cơ sở dữ liệu và phát triển web. Các nội dung cần tiếp tục hoàn thiện là đồng bộ thời gian và độ mới dữ liệu, thống nhất trạng thái giữa thiết bị với dashboard, bổ sung xác nhận lệnh, xử lý lỗi, kiểm soát truy cập và kiểm thử dài ngày. Những cải tiến này là cơ sở để mở rộng từ mô hình học tập sang một hệ thống có phạm vi sử dụng lớn hơn.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems. GPIO and RTC GPIO, ESP-IDF Programming Guide. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/gpio.html

[2] ESPHome. MQTT Client Component. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://esphome.io/components/mqtt/

[3] ESPHome. DHT Temperature and Humidity Sensor. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://esphome.io/components/sensor/dht/

[4] MQTT.org. MQTT The Standard for IoT Messaging. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://mqtt.org/

[5] Express. Routing. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://expressjs.com/en/guide/routing/

[6] React. Quick Start. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://react.dev/learn

[7] Ollama. Generate a chat message. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://docs.ollama.com/api/chat

[8] Nhóm sinh viên D18CNPM5. Mã nguồn dự án nhà thông minh: nha-thong-minh-backend/index.js, nha-thong-minh-dashboard/src/App.jsx; cấu hình ESPHome và ảnh mô hình. Tài liệu nội bộ của đề tài, 2026.

[9] Cơ sở dữ liệu nha_thong_minh. Hai phiên dữ liệu ngày 01/09/2026 và 08/09/2026; kết quả kiểm tra dịch vụ ngày 15/10/2026. Dữ liệu thực nghiệm nội bộ của nhóm.

[10] OASIS. MQTT Version 3.1.1, OASIS Standard, 29/10/2014. Các mục QoS, retained message và topic filter. Truy cập ngày 15/10/2026. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html

[11] Docker. Port publishing and mapping. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://docs.docker.com/engine/network/port-publishing/

[12] Vite. Server Options. Tài liệu trực tuyến. Truy cập ngày 15/10/2026. https://vite.dev/config/server-options

PHỤ LỤC A CẤU HÌNH ESPHOME

Cấu hình dưới đây thể hiện các chân, topic và hành động của phiên bản mô hình sử dụng trong báo cáo. Các giá trị xác thực được tham chiếu qua secrets.yaml. Việc tách secrets không thay đổi thuật toán PIR, LED, relay, còi và chế độ thủ công.

```
# Cau hinh ESPHome dua tren phien ban nhom cung cap
# Thong tin xac thuc duoc tach sang secrets.yaml
esphome:
  name: cam-bien-phong-khach
  friendly_name: cam-bien-phong-khach
esp32:
  variant: esp32
  framework:
    type: esp-idf
logger:
mqtt:
  broker: 192.168.0.102
  port: 1883
  discovery: false
  password: !secret mqtt_password
api:
  encryption:
    key: !secret api_encryption_key
ota:
  - platform: esphome
    password: !secret ota_password
wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_password
  ap:
    ssid: cam-bien-phong- Fallback Hotspot
    password: !secret fallback_ap_password
captive_portal:
globals:
  - id: che_do_thu_cong
    type: bool
    restore_value: no
    initial_value: 'false'
sensor:
  - platform: dht
    id: sensor_dht_1
    pin: GPIO4
    humidity:
      name: Độ ẩm phòng
      id: sensor_dht_1_humidity
      state_topic: nha/phongkhach/doam/state
    temperature:
      name: Nhiệt độ phòng
      id: sensor_dht_1_temperature
      state_topic: nha/phongkhach/nhietdo/state
    model: DHT11
binary_sensor:
  - platform: gpio
    name: Chuyển động phòng khách
    id: binary_sensor_gpio_1
    pin: GPIO27
    state_topic: nha/phongkhach/chuyendong/state
    on_press:
```

```
      then:
        - if:
            condition:
              lambda: 'return !id(che_do_thu_cong);'
            then:
              - switch.turn_on: den_led_chuyendong
              - switch.turn_on: chuong_cua
              - delay: 500ms
              - switch.turn_off: chuong_cua
              - if:
                  condition:
                    binary_sensor.is_on: binary_sensor_gpio_2
                  then:
                    - switch.turn_on: switch_gpio_1
    on_release:
      then:
        - if:
            condition:
              lambda: 'return !id(che_do_thu_cong);'
            then:
              - switch.turn_off: den_led_chuyendong
              - switch.turn_off: switch_gpio_1
  - platform: gpio
    name: Ánh sáng phòng khách
    id: binary_sensor_gpio_2
    pin: GPIO34
    state_topic: nha/phongkhach/anhsang/state
switch:
  - platform: gpio
    name: Đèn LED chuyển động
    id: den_led_chuyendong
    pin: GPIO25
    state_topic: nha/phongkhach/den/state
    command_topic: nha/phongkhach/den/set
  - platform: gpio
    name: Relay phòng khách
    id: switch_gpio_1
    state_topic: nha/phongkhach/relay/state
    command_topic: nha/phongkhach/relay/set
    pin:
      number: GPIO26
      inverted: true
  - platform: gpio
    name: Chuông cửa
    id: chuong_cua
    pin: GPIO33
    state_topic: nha/phongkhach/coi/state
    command_topic: nha/phongkhach/coi/set
  - platform: template
    name: Chế độ thủ công
    id: cong_tac_thu_cong
    optimistic: true
    state_topic: nha/phongkhach/thucong/state
    command_topic: nha/phongkhach/thucong/set
    turn_on_action:
      - globals.set:
          id: che_do_thu_cong
          value: 'true'
    turn_off_action:
      - globals.set:
          id: che_do_thu_cong
          value: 'false'
```

PHỤ LỤC B HƯỚNG DẪN CHẠY VÀ ĐỐI CHIẾU

Khởi động các dịch vụ

Kiểm tra MySQL80 đang chạy và cơ sở dữ liệu nha_thong_minh có hai bảng cam_bien_data, thoi_quen_nguoi_dung. Kiểm tra file .env của backend chứa cấu hình kết nối DB_HOST, DB_USER, DB_PASSWORD và DB_NAME. Không đưa giá trị mật khẩu vào log hoặc tài liệu chia sẻ.

```
docker start mosquitto

# Tai thu muc nha-thong-minh-backend
node index.js

# Tai thu muc nha-thong-minh-dashboard
npm run dev

# Dich vu AI
ollama serve
ollama list
```

Trên Windows PowerShell, nếu npm.ps1 bị chính sách thực thi chặn, dùng npm.cmd cho cùng lệnh. Khi Ollama đã chạy thì không cần mở thêm một tiến trình serve. Danh sách model cần có llama3.1:latest để phù hợp với cấu hình gọi hiện tại.

Địa chỉ dùng khi chạy trên máy chủ

Dịch vụ	Địa chỉ hoặc cổng
Dashboard phát triển	http://localhost:5173
REST API	http://localhost:3000
WebSocket	ws://localhost:8081
MQTT backend	mqtt://localhost:1883
Ollama	http://localhost:11434
MQTT phía ESP32	192.168.0.102:1883 trong cấu hình của nhóm

Ví dụ nội dung yêu cầu và phản hồi

Các ví dụ JSON dưới mô tả giao ước API. Yêu cầu relay có thể làm thay đổi thiết bị khi hệ thống đang hoạt động; chỉ gửi khi chủ động kiểm tra luồng điều khiển.

```
POST /api/relay
{"trang_thai":"ON"}
```

```
POST /api/che-do-thu-cong
{"trang_thai":"OFF"}

POST /api/chatbot
{"cau_hoi":"What are the temperature and humidity readings?"}

# Phan hoi hoi dap da ghi nhan
{"tra_loi":"The current temperature is 29.9°C, and the humidity is at 83%."}
```

Truy vấn đối chiếu tập dữ liệu lịch sử

Các truy vấn dùng mốc trước ngày 15/10 để tách tập dữ liệu lịch sử khỏi các bản tin được nhận lại trong lần khởi động kiểm tra. Với phiên đo mới, cần chọn cửa sổ thời gian tương ứng và kiểm tra nguồn gốc bản tin trước khi thống kê.

```
SELECT topic, COUNT(*) AS so_hang
FROM cam_bien_data
WHERE thoi_gian < '2026-10-15'
GROUP BY topic;

SELECT topic,
       MIN(CAST(gia_tri AS DECIMAL(8,2))) AS nho_nhat,
       MAX(CAST(gia_tri AS DECIMAL(8,2))) AS lon_nhat,
       AVG(CAST(gia_tri AS DECIMAL(8,2))) AS trung_binh
FROM cam_bien_data
WHERE thoi_gian < '2026-10-15'
  AND topic IN ('nha/phongkhach/nhietdo/state',
                'nha/phongkhach/doam/state')
GROUP BY topic;
```

Các lưu ý khi mở rộng cấu hình

Nếu dùng MQTT cùng native API ESPHome nhưng không có native API client kết nối, cần xem xét cấu hình reboot_timeout theo tài liệu MQTT của ESPHome [2]. Khi sửa cực tính ánh sáng, phải kiểm tra cả nhãn frontend và điều kiện điều khiển relay. Khi triển khai từ điện thoại, thay localhost bằng địa chỉ máy chủ phù hợp trong LAN và cấu hình quyền truy cập cần thiết.

Để đo độ trễ điều khiển trong lần phát triển tiếp theo, cần đồng bộ thời gian, thêm mã lệnh và lưu riêng các mốc gửi lệnh, nhận lệnh, cập nhật đầu ra, nhận trạng thái. Để đánh giá cảm biến, cần ghi đồng thời số đọc của thiết bị chuẩn. Để đánh giá tiết kiệm điện, cần đo công suất và thời lượng tải ở một cấu hình đối chứng tương ứng.